

幂级数与拉格朗日插值

Power Series and Lagrange Interpolation

Puppy

2023 年 12 月 22 日

- 由于讲课人很菜，所以无法做到深入讲解。

- 由于讲课人很菜，所以无法做到深入讲解。
- 不太知道该怎么讲，所以课件中包含大量例题，在例题中感受那股劲（逃。

目录

1. 牛顿二项式定理
2. 多项式基础操作
3. 生成函数引入
4. 常见数列的生成函数
5. OGF、EGF 与 PGF
6. 生成函数杂项
7. 生成函数杂题
8. 符号化方法入门
9. 组合计数例题
10. SOS DP
11. 快速沃尔什变换
12. 集合幂级数
13. 拉格朗日插值

牛顿二项式定理

二项式系数

定义 1.0.1 (二项式系数)

$$\binom{n}{m} = \begin{cases} \frac{n^{\underline{m}}}{m!} & m \geq 0 \\ 0 & m < 0 \end{cases}$$

其中 $n \in \mathbb{R}, m \in \mathbb{Z}$ 。

二项式系数

定义 1.0.1 (二项式系数)

$$\binom{n}{m} = \begin{cases} \frac{n^{\underline{m}}}{m!} & m \geq 0 \\ 0 & m < 0 \end{cases}$$

其中 $n \in \mathbb{R}, m \in \mathbb{Z}$ 。

- 其中 $n^{\underline{m}}$ 即 n 的 m 次下降幂, $n^{\underline{m}} = n \times (n-1) \times \dots \times (n-m+1)$ 。

例如: $1.5^{\underline{3}} = 1.5 \times 0.5 \times (-0.5) = -\frac{3}{8}$ 。

牛顿二项式定理

定义 1.0.2 (牛顿二项式定理)

$$(1+z)^r = \sum_k \binom{r}{k} z^k$$

其中 $r \in \mathbb{R}, z \in \mathbb{C}$ 。

牛顿二项式定理

定义 1.0.2 (牛顿二项式定理)

$$(1+z)^r = \sum_k \binom{r}{k} z^k$$

其中 $r \in \mathbb{R}, z \in \mathbb{C}$ 。

- 证明:

设 $f(z) = (1+z)^r$, 根据泰勒展开:

$$f(z) = \sum_{k \geq 0} \frac{f^{(k)}(0)}{k!} z^k$$

$$\because f^{(k)}(z) = r^{\underline{k}} (1+z)^{r-k}$$

$$\therefore f(z) = \sum_{k \geq 0} \frac{r^{\underline{k}}}{k!} z^k = \sum_{k \geq 0} \binom{r}{k} z^k$$

多项式基础操作

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$, 即找到一个多项式 $G(x)$, 满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。
- 多项式 $F(x)$ 在模 x^m 意义下有逆的充要条件为 $[x^0]F(x) \neq 0$ ，必要性是显然的，若 0 次项为零，乘任何多项式后 0 次项都不为 1。

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。
- 多项式 $F(x)$ 在模 x^m 意义下有逆的充要条件为 $[x^0]F(x) \neq 0$ ，必要性是显然的，若 0 次项为零，乘任何多项式后 0 次项都不为 1。
- 充分性可以考虑如下构造：

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$, 即找到一个多项式 $G(x)$, 满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。
- 多项式 $F(x)$ 在模 x^m 意义下有逆的充要条件为 $[x^0]F(x) \neq 0$, 必要性是显然的, 若 0 次项为零, 乘任何多项式后 0 次项都不为 1。
- 充分性可以考虑如下构造:
- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$, 则:

$$f_0 g_0 = 1$$

$$\sum_{0 \leq x \leq n} f_x g_{n-x} = 0 \quad n > 0$$

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$, 即找到一个多项式 $G(x)$, 满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。
- 多项式 $F(x)$ 在模 x^m 意义下有逆的充要条件为 $[x^0]F(x) \neq 0$, 必要性是显然的, 若 0 次项为零, 乘任何多项式后 0 次项都不为 1。
- 充分性可以考虑如下构造:
- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$, 则:

$$f_0 g_0 = 1$$

$$\sum_{0 \leq x \leq n} f_x g_{n-x} = 0 \quad n > 0$$

- 也就是说 $g_0 = f_0^{-1}$, 而对于 $n > 0$, $g_n = f_0^{-1} - \sum_{0 \leq x < n} g_x f_{n-x}$ 。

多项式求逆

- 假如我们要求 $F^{-1}(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $F(x)G(x) \equiv 1 \pmod{x^m}$ 。
- 多项式 $F(x)$ 在模 x^m 意义下有逆的充要条件为 $[x^0]F(x) \neq 0$ ，必要性是显然的，若 0 次项为零，乘任何多项式后 0 次项都不为 1。
- 充分性可以考虑如下构造：
- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ ，则：

$$f_0 g_0 = 1$$

$$\sum_{0 \leq x \leq n} f_x g_{n-x} = 0 \quad n > 0$$

- 也就是说 $g_0 = f_0^{-1}$ ，而对于 $n > 0$ ， $g_n = f_0^{-1} - \sum_{0 \leq x < n} g_x f_{n-x}$ 。
- 由于在模 x^m 意义下，只需算到 g_{m-1} 即可，我们就在 $\Theta(m^2)$ 的时间复杂度内求出了 $G(x)$ 。

- 现在我们要要求 $\ln F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $G(x) \equiv \ln F(x) \pmod{x^m}$ 。

- 现在我们要要求 $\ln F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $G(x) \equiv \ln F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 有一个限制，即 $[x^0]F(x) = 1$ ，因为 $[x^0]G(x) = \ln[x^0]F(x)$ ，而当 $[x^0]F(x) \neq 1$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

- 现在我们要要求 $\ln F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $G(x) \equiv \ln F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 有一个限制，即 $[x^0]F(x) = 1$ ，因为 $[x^0]G(x) = \ln[x^0]F(x)$ ，而当 $[x^0]F(x) \neq 1$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。
-

$$G(x) = \ln F(x)$$

- 现在我们要要求 $\ln F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $G(x) \equiv \ln F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 有一个限制，即 $[x^0]F(x) = 1$ ，因为 $[x^0]G(x) = \ln[x^0]F(x)$ ，而当 $[x^0]F(x) \neq 1$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

- $$G(x) = \ln F(x)$$

- $$G'(x) = \frac{F'(x)}{F(x)}$$

- 现在我们要要求 $\ln F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $G(x) \equiv \ln F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 有一个限制，即 $[x^0]F(x) = 1$ ，因为 $[x^0]G(x) = \ln[x^0]F(x)$ ，而当 $[x^0]F(x) \neq 1$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

- $$G(x) = \ln F(x)$$

- $$G'(x) = \frac{F'(x)}{F(x)}$$

- $$G'(x)F(x) = F'(x)$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

-

$$g_0 = 0$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

-

$$g_0 = 0$$

-

$$\sum_{0 \leq x, y \leq n, x+y=n} g_{x+1}(x+1)f_y = f_{n+1}(n+1) \quad n \geq 0$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

-

$$g_0 = 0$$

-

$$\sum_{0 \leq x, y \leq n, x+y=n} g_{x+1}(x+1)f_y = f_{n+1}(n+1) \quad n \geq 0$$

- 与多项式求逆类似，根据第二个等式可以从 g_0 一直递推到 g_{m-1} ，于是可以在 $\Theta(m^2)$ 的时间复杂度内求出 $G(x)$ 。

- 接着我们要求 $\exp F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $\ln G(x) \equiv F(x) \pmod{x^m}$ 。

- 接着我们要求 $\exp F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $\ln G(x) \equiv F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 也有一个限制，即 $[x^0]F(x) = 0$ ，由于 $[x^0]G(x) = \exp([x^0]F(x))$ ，当 $[x^0]F(x) \neq 0$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

多项式 exp

- 接着我们要求 $\exp F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $\ln G(x) \equiv F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 也有一个限制，即 $[x^0]F(x) = 0$ ，由于 $[x^0]G(x) = \exp([x^0]F(x))$ ，当 $[x^0]F(x) \neq 0$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。
-

$$\ln G(x) = F(x)$$

多项式 exp

- 接着我们要求 $\exp F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $\ln G(x) \equiv F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 也有一个限制，即 $[x^0]F(x) = 0$ ，由于 $[x^0]G(x) = \exp([x^0]F(x))$ ，当 $[x^0]F(x) \neq 0$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

- $$\ln G(x) = F(x)$$

- $$\frac{G'(x)}{G(x)} = F'(x)$$

多项式 exp

- 接着我们要求 $\exp F(x) \pmod{x^m}$ ，即找到一个多项式 $G(x)$ ，满足 $\ln G(x) \equiv F(x) \pmod{x^m}$ 。
- 此处对 $F(x)$ 也有一个限制，即 $[x^0]F(x) = 0$ ，由于 $[x^0]G(x) = \exp([x^0]F(x))$ ，当 $[x^0]F(x) \neq 0$ 时无法在模质数意义下计算 $[x^0]G(x)$ 。

- $$\ln G(x) = F(x)$$

- $$\frac{G'(x)}{G(x)} = F'(x)$$

- $$F'(x)G(x) = G'(x)$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。
-

$$g_0 = 1$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

-

$$g_0 = 1$$

-

$$\sum_{0 \leq x, y \leq n, x+y=n} f_{x+1}(x+1)g_y = g_{n+1}(n+1) \quad n \geq 0$$

- 设 $f_n = [x^n]F(x)$, $g_n = [x^n]G(x)$ 。

- $$g_0 = 1$$

- $$\sum_{0 \leq x, y \leq n, x+y=n} f_{x+1}(x+1)g_y = g_{n+1}(n+1) \quad n \geq 0$$

- 与前面两个操作一样，从 g_0 递推到 g_{m-1} 即可，时间复杂度 $\Theta(m^2)$ 。

生成函数引入

- 一个序列 a 的生成函数 $F(x)$ 一般定义为：

$$F(x) = \sum_n a_n k_n(x)$$

- 一个序列 a 的生成函数 $F(x)$ 一般定义为：

$$F(x) = \sum_n a_n k_n(x)$$

- 其中 $k_n(x)$ 被称为核函数， a_n 称为 $F(x)$ 的第 n 项系数，可以用 $[k_n(x)]F(x)$ 表示。

生成函数引入

- 一个序列 a 的生成函数 $F(x)$ 一般定义为：

$$F(x) = \sum_n a_n k_n(x)$$

- 其中 $k_n(x)$ 被称为核函数， a_n 称为 $F(x)$ 的第 n 项系数，可以用 $[k_n(x)]F(x)$ 表示。
- 当 $k_n(x) = x^n$ 时为普通型生成函数，当 $k_n(x) = \frac{x^n}{n!}$ 时为指数型生成函数。

生成函数引入

- 一个序列 a 的生成函数 $F(x)$ 一般定义为：

$$F(x) = \sum_n a_n k_n(x)$$

- 其中 $k_n(x)$ 被称为核函数， a_n 称为 $F(x)$ 的第 n 项系数，可以用 $[k_n(x)]F(x)$ 表示。
- 当 $k_n(x) = x^n$ 时为普通型生成函数，当 $k_n(x) = \frac{x^n}{n!}$ 时为指数型生成函数。
- 一些对序列的变换可以用生成函数的运算表示，从而进行一些推导，达到优化复杂度的目的。

常见数列的生成函数

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}.$

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}$ 。
- 证明：

$$xF(x) + 1 = F(x)$$

$$F(x) = \frac{1}{1-x}$$

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}$ 。
- 证明：

$$xF(x) + 1 = F(x)$$

$$F(x) = \frac{1}{1-x}$$

- 同理：

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}$ 。
- 证明：

$$xF(x) + 1 = F(x)$$

$$F(x) = \frac{1}{1-x}$$

- 同理：
- $F(x) = \sum_{n \geq 0} a^n x^n \rightarrow F(x) = \frac{1}{1-ax}$ 。

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}$ 。
- 证明：

$$xF(x) + 1 = F(x)$$

$$F(x) = \frac{1}{1-x}$$

- 同理：
- $F(x) = \sum_{n \geq 0} a^n x^n \rightarrow F(x) = \frac{1}{1-ax}$ 。
- $F(x) = \sum_{n \geq 0} x^{nk} \rightarrow F(x) = \frac{1}{1-x^k}$ 。

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} x^n \rightarrow \frac{1}{1-x}.$
- 证明:

$$xF(x) + 1 = F(x)$$

$$F(x) = \frac{1}{1-x}$$

- 同理:
- $F(x) = \sum_{n \geq 0} a^n x^n \rightarrow F(x) = \frac{1}{1-ax}.$
- $F(x) = \sum_{n \geq 0} x^{nk} \rightarrow F(x) = \frac{1}{1-x^k}.$
- $F(x) = \sum_{n \geq 0} a^n x^{nk} \rightarrow F(x) = \frac{1}{1-ax^k}.$

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} \binom{n+m}{m} x^n \rightarrow F(x) = \frac{1}{(1-x)^{m+1}}.$

一些基础的生成函数

- $F(x) = \sum_{n \geq 0} \binom{n+m}{m} x^n \rightarrow F(x) = \frac{1}{(1-x)^{m+1}}$ 。
- 证明：

$$\sum_{n \geq 0} x^n = \frac{1}{1-x}$$

两边同求 m 次导，可得：

$$\sum_{n \geq 0} (n+m)^m x^n = \frac{m!}{(1-x)^{m+1}}$$

$$\sum_{n \geq 0} \binom{n+m}{m} x^n = \frac{1}{(1-x)^{m+1}}$$

一些基础的生成函数

- $F(x) = -\ln(1 - x^2) \rightarrow F(x) = \sum_{n \geq 1} \frac{x^{2n}}{n}$

一些基础的生成函数

- $F(x) = -\ln(1 - x^2) \rightarrow F(x) = \sum_{n \geq 1} \frac{x^{2n}}{n}$
- 证明:

$$F(x) = -\ln(1 - x^2)$$

两边同时求导, 得:

$$F'(x) = \frac{2x^{2-1}}{1 - x^2}$$

$$F(x) = \int x^{2-1} \frac{2}{1 - x^2} = \sum_{n \geq 1} \frac{2x^{2n}}{2n} = \sum_{n \geq 1} \frac{x^{2n}}{n}$$

Fibonacci 数列的生成函数

- $f_0 = 0, f_1 = 1;$
 $f_n = f_{n-1} + f_{n-2}. \quad (n > 1)$

Fibonacci 数列的生成函数

- $f_0 = 0, f_1 = 1;$
 $f_n = f_{n-1} + f_{n-2}. \quad (n > 1)$
- 设 $F(x) = \sum_{n \geq 0} f_n x^n$ 。

Fibonacci 数列的生成函数

- $f_0 = 0, f_1 = 1;$
 $f_n = f_{n-1} + f_{n-2}. \quad (n > 1)$
- 设 $F(x) = \sum_{n \geq 0} f_n x^n$.
-

$$xF(x) + x^2F(x) + x = F(x) \rightarrow F(x) = \frac{x}{1 - x - x^2}$$

Fibonacci 数列的生成函数

- $f_0 = 0, f_1 = 1;$
 $f_n = f_{n-1} + f_{n-2}. \quad (n > 1)$

- 设 $F(x) = \sum_{n \geq 0} f_n x^n$ 。

-

$$xF(x) + x^2F(x) + x = F(x) \rightarrow F(x) = \frac{x}{1 - x - x^2}$$

- 于是我们得到了 Fibonacci 数列的生成函数。

- 发现我们并不能快速得到 $\frac{x}{1-x-x^2}$ 的第 n 项系数。

Fibonacci 数列通项公式推导

- 发现我们并不能快速得到 $\frac{x}{1-x-x^2}$ 的第 n 项系数。
- 但是我们发现形如 $\frac{A}{1-ax}$ 的生成函数的第 n 项系数，我们能够快速得到。

Fibonacci 数列通项公式推导

- 发现我们并不能快速得到 $\frac{x}{1-x-x^2}$ 的第 n 项系数。
- 但是我们发现形如 $\frac{A}{1-ax}$ 的生成函数的第 n 项系数，我们能够快速得到。
- 观察 Fibonacci 数列的生成函数，设：

Fibonacci 数列通项公式推导

- 发现我们并不能快速得到 $\frac{x}{1-x-x^2}$ 的第 n 项系数。
- 但是我们发现形如 $\frac{A}{1-ax}$ 的生成函数的第 n 项系数，我们能够快速得到。
- 观察 Fibonacci 数列的生成函数，设：
-

$$F(x) = \frac{x}{1-x-x^2} = \frac{A}{1-ax} + \frac{B}{1-bx}$$

Fibonacci 数列通项公式推导

- 发现我们并不能快速得到 $\frac{x}{1-x-x^2}$ 的第 n 项系数。
- 但是我们发现形如 $\frac{A}{1-ax}$ 的生成函数的第 n 项系数，我们能够快速得到。
- 观察 Fibonacci 数列的生成函数，设：

$$F(x) = \frac{x}{1-x-x^2} = \frac{A}{1-ax} + \frac{B}{1-bx}$$

$$\frac{x}{1-x-x^2} = \frac{A+B-Abx-Bax}{(1-ax)(1-bx)}$$

Fibonacci 数列通项公式推导

•

$$\begin{cases} A + B = 0 \\ Ab + Ba = -1 \\ a + b = 1 \\ ab = -1 \end{cases} \quad \begin{cases} a = \frac{1 + \sqrt{5}}{2} \\ b = \frac{1 - \sqrt{5}}{2} \\ A = \frac{\sqrt{5}}{5} \\ B = -\frac{\sqrt{5}}{5} \end{cases}$$

Fibonacci 数列通项公式推导

- $$\begin{cases} A + B = 0 \\ Ab + Ba = -1 \\ a + b = 1 \\ ab = -1 \end{cases} \quad \begin{cases} a = \frac{1 + \sqrt{5}}{2} \\ b = \frac{1 - \sqrt{5}}{2} \\ A = \frac{\sqrt{5}}{5} \\ B = -\frac{\sqrt{5}}{5} \end{cases}$$

- $$[x^n]F(x) = A \cdot a^n + B \cdot b^n$$

Fibonacci 数列通项公式推导

- $$\begin{cases} A + B = 0 \\ Ab + Ba = -1 \\ a + b = 1 \\ ab = -1 \end{cases} \quad \begin{cases} a = \frac{1 + \sqrt{5}}{2} \\ b = \frac{1 - \sqrt{5}}{2} \\ A = \frac{\sqrt{5}}{5} \\ B = -\frac{\sqrt{5}}{5} \end{cases}$$

- $$[x^n]F(x) = A \cdot a^n + B \cdot b^n$$

- $$f_n = \frac{\sqrt{5}}{5} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{\sqrt{5}}{5} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

Fibonacci 数列通项公式推导

- $$\begin{cases} A + B = 0 \\ Ab + Ba = -1 \\ a + b = 1 \\ ab = -1 \end{cases} \quad \begin{cases} a = \frac{1 + \sqrt{5}}{2} \\ b = \frac{1 - \sqrt{5}}{2} \\ A = \frac{\sqrt{5}}{5} \\ B = -\frac{\sqrt{5}}{5} \end{cases}$$

- $$[x^n]F(x) = A \cdot a^n + B \cdot b^n$$

- $$f_n = \frac{\sqrt{5}}{5} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{\sqrt{5}}{5} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

- 于是我们得到了 Fibonacci 数列的通项公式。

Catalan 数的生成函数

- $C_0 = 1;$
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k}. \quad (n > 0)$

Catalan 数的生成函数

- $C_0 = 1;$
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k}. \quad (n > 0)$
- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。

Catalan 数的生成函数

- $C_0 = 1$;
$$C_{n+1} = \sum_{k=0}^n C_k C_{n-k}. \quad (n > 0)$$
- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。
- 由于其递推式为卷积的形式：

Catalan 数的生成函数

- $C_0 = 1$;
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k} \quad (n > 0)$
- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。
- 由于其递推式为卷积的形式：
-

$$F^2(x) \cdot x + 1 = F(x)$$

Catalan 数的生成函数

- $C_0 = 1$;
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k} \quad (n > 0)$

- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。
- 由于其递推式为卷积的形式：

$$F^2(x) \cdot x + 1 = F(x)$$

- $$F(x) = \frac{1 \pm \sqrt{1-4x}}{2x} = \frac{2}{1 \mp \sqrt{1-4x}}$$

Catalan 数的生成函数

- $C_0 = 1$;
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k} \quad (n > 0)$

- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。
- 由于其递推式为卷积的形式：

$$F^2(x) \cdot x + 1 = F(x)$$

$$F(x) = \frac{1 \pm \sqrt{1-4x}}{2x} = \frac{2}{1 \mp \sqrt{1-4x}}$$

- 代入 $x = 0$ ，其中一个根可以求出 C_0 ，一个根分母为 0，舍去。

Catalan 数的生成函数

- $C_0 = 1$;
 $C_{n+1} = \sum_{k=0}^n C_k C_{n-k} \quad (n > 0)$

- 设 $F(x) = \sum_{n \geq 0} C_n x^n$ 。
- 由于其递推式为卷积的形式：

$$F^2(x) \cdot x + 1 = F(x)$$

$$F(x) = \frac{1 \pm \sqrt{1-4x}}{2x} = \frac{2}{1 \mp \sqrt{1-4x}}$$

- 代入 $x = 0$ ，其中一个根可以求出 C_0 ，一个根分母为 0，舍去。
- 于是得到 $F(x) = \frac{1 - \sqrt{1-4x}}{2x}$ 。

Catalan 数通项公式推导

- 根据牛顿二项式定理：

Catalan 数通项公式推导

- 根据牛顿二项式定理：

-

$$(1 - 4x)^{\frac{1}{2}} = \sum_{n \geq 0} \binom{\frac{1}{2}}{n} (-4x)^n = 1 + \sum_{n \geq 1} \binom{\frac{1}{2}}{n} (-4x)^n$$

Catalan 数通项公式推导

- 根据牛顿二项式定理：

$$(1 - 4x)^{\frac{1}{2}} = \sum_{n \geq 0} \binom{\frac{1}{2}}{n} (-4x)^n = 1 + \sum_{n \geq 1} \binom{\frac{1}{2}}{n} (-4x)^n$$

$$\begin{aligned} \binom{\frac{1}{2}}{n} &= \frac{\frac{1}{2} \times (-\frac{1}{2}) \times \cdots \times (\frac{3-2n}{2})}{n!} \\ &= \frac{(-1)^{n-1} (2n-3)!!}{2^n n!} \\ &= \frac{(-1)^{n-1} (2n-2)!}{2^n n! (2n-2)!!} \\ &= \frac{(-1)^{n-1} (2n-2)!}{2^{2n-1} (n-1)! n!} \end{aligned}$$

Catalan 数通项公式推导

- 代入得：

Catalan 数通项公式推导

- 代入得：
-

$$\begin{aligned}(1-4x)^{\frac{1}{2}} &= 1 - \sum_{n \geq 1} \frac{(2n-2)!}{n!(n-1)!} 2x^n \\ &= 1 - \sum_{n \geq 1} \binom{2n-1}{n} \frac{2x^n}{2n-1}\end{aligned}$$

Catalan 数通项公式推导

- 代入得：

$$\begin{aligned}(1-4x)^{\frac{1}{2}} &= 1 - \sum_{n \geq 1} \frac{(2n-2)!}{n!(n-1)!} 2x^n \\ &= 1 - \sum_{n \geq 1} \binom{2n-1}{n} \frac{2x^n}{2n-1}\end{aligned}$$

$$\begin{aligned}F(x) &= \frac{1}{2x} \sum_{n \geq 1} \binom{2n-1}{n} \frac{2x^n}{2n-1} \\ &= \sum_{n \geq 1} \binom{2n-1}{n} \frac{x^{n-1}}{2n-1} \\ &= \sum_{n \geq 0} \binom{2n+1}{n+1} \frac{x^n}{2n+1} \\ &= \sum_{n \geq 0} \binom{2n}{n} \frac{1}{n+1} x^n\end{aligned}$$

- $C_n = \binom{2n}{n} \frac{1}{n+1}$

- $C_n = \binom{2n}{n} \frac{1}{n+1}$
- 于是我们得到了 Catalan 数的通项公式。

OGF、EGF 与 PGF

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 其中 a 可以是有穷序列, 也可以是无穷序列。

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 其中 a 可以是有穷序列, 也可以是无穷序列。
- 考虑两个序列 a, b 的生成函数 $F(x), G(x)$, 那么生成函数加法运算定义为:

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 其中 a 可以是有穷序列, 也可以是无穷序列。
- 考虑两个序列 a, b 的生成函数 $F(x), G(x)$, 那么生成函数加法运算定义为:
-

$$F(x) \pm G(x) = \sum_{n \geq 0} (a_n \pm b_n) x^n$$

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 其中 a 可以是有穷序列, 也可以是无穷序列。
- 考虑两个序列 a, b 的生成函数 $F(x), G(x)$, 那么生成函数加法运算定义为:
-

$$F(x) \pm G(x) = \sum_{n \geq 0} (a_n \pm b_n) x^n$$

- 生成函数乘法运算定义为:

OGF 的定义

- OGF, 全称 ordinary generating function, 普通生成函数。
- 一个序列 a 的 OGF 定义为:
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 其中 a 可以是有穷序列, 也可以是无穷序列。
- 考虑两个序列 a, b 的生成函数 $F(x), G(x)$, 那么生成函数加法运算定义为:
-

$$F(x) \pm G(x) = \sum_{n \geq 0} (a_n \pm b_n) x^n$$

- 生成函数乘法运算定义为:
-

$$F(x)G(x) = \sum_{n \geq 0} x^n \sum_{i=0}^n a_i b_{n-i}$$

- OGF 一般用来处理无标号体系。

- OGF 一般用来处理无标号体系。
- 考虑若干个无标号的物品，每个物品有重量。

- OGF 一般用来处理无标号体系。
- 考虑若干个无标号的物品，每个物品有重量。
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- OGF 一般用来处理无标号体系。
- 考虑若干个无标号的物品，每个物品有重量。
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 将 F 看作一个背包， a_n 表示在背包中选出总重量为 n 的物品有多少种方案。

- OGF 一般用来处理无标号体系。
- 考虑若干个无标号的物品，每个物品有重量。
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 将 F 看作一个背包， a_n 表示在背包中选出总重量为 n 的物品有多少种方案。
- $F(x)G(x)$ 就相当于合并两个背包。

- OGF 一般用来处理无标号体系。
- 考虑若干个无标号的物品，每个物品有重量。
-

$$F(x) = \sum_{n \geq 0} a_n x^n$$

- 将 F 看作一个背包， a_n 表示在背包中选出总重量为 n 的物品有多少种方案。
- $F(x)G(x)$ 就相当于合并两个背包。
- 常用的 OGF 在前一章已给出，这里不再赘述。

- 有一副超级扑克，对于每个正合数 p ，恰好有四种花色：黑桃 p ，红桃 p ，梅花 p 和方块 p （分别用 pS, pH, pC, pD 表示）。
- 有些牌已经丢失，给出丢失的牌，在剩下的牌中四种花色每种选出一张，求选出牌的点数之和为 n 的方案数。
- 题目给出 a, b ，求 $n = a, a + 1, \dots, b$ 时的答案。
- 设丢失的牌数量为 c 。
- 多测，不超过 25 组数据，最多有一组数据满足 $a = 1, b = 5 \times 10^4, c \leq 10^4$ ，其它数据满足 $1 \leq a \leq b \leq 100, 0 \leq c \leq 10$ 。

- 发现是一个背包板子。

- 发现是一个背包板子。
- 对于每种花色建立生成函数 $F(x) = \sum_{n \geq 1} a_n x^n$ ，其中 a_n 表示当前花色，点数为 n 的牌有多少张。

- 发现是一个背包板子。
- 对于每种花色建立生成函数 $F(x) = \sum_{n \geq 1} a_n x^n$ ，其中 a_n 表示当前花色，点数为 n 的牌有多少张。
- 将四个生成函数乘起来，输出 $a, a + 1, \dots, b$ 项的系数即可。

- 发现是一个背包板子。
- 对于每种花色建立生成函数 $F(x) = \sum_{n \geq 1} a_n x^n$ ，其中 a_n 表示当前花色，点数为 n 的牌有多少张。
- 将四个生成函数乘起来，输出 $a, a+1, \dots, b$ 项的系数即可。
- 利用 FFT 加速卷积，时间复杂度 $\Theta(n \log n)$ 。

- 付公主有一个可爱的背包 qwq。
- 这个背包最多可以装 10^5 大小的东西。
- 付公主有 n 种商品，她要准备出摊了。
- 每种商品体积为 v_i ，都有无限件。
- 给定 m ，对于 $s \in [1, m]$ ，请你回答用这些商品恰好装 s 体积的方案数，答案对 998244353 取模。
- $1 \leq n, m \leq 10^5, 1 \leq v_i \leq m$ 。

P4389 付公主的背包

- 又是一个背包板子。

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

P4389 付公主的背包

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

- 则 $F(x)$ 的第 m 项系数就是 $s = m$ 时的答案。

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

- 则 $F(x)$ 的第 m 项系数就是 $s = m$ 时的答案。
-

$$F(x) = \prod_{i=1}^n \frac{1}{1 - x^{v_i}}$$

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

- 则 $F(x)$ 的第 m 项系数就是 $s = m$ 时的答案。
-

$$F(x) = \prod_{i=1}^n \frac{1}{1 - x^{v_i}}$$

•

$$F(x) = \exp \left(\sum_{i=1}^n -\ln(1 - x^{v_i}) \right)$$

P4389 付公主的背包

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

- 则 $F(x)$ 的第 m 项系数就是 $s = m$ 时的答案。
-

$$F(x) = \prod_{i=1}^n \frac{1}{1 - x^{v_i}}$$

•

$$F(x) = \exp \left(\sum_{i=1}^n -\ln(1 - x^{v_i}) \right)$$

•

$$F(x) = \exp \left(\sum_{i=1}^n \sum_{j \geq 0} \frac{x^{v_{ij}}}{j} \right)$$

P4389 付公主的背包

- 又是一个背包板子。
- 设：

$$F(x) = \prod_{i=1}^n \left(\sum_{z \geq 0} x^{zv_i} \right)$$

- 则 $F(x)$ 的第 m 项系数就是 $s = m$ 时的答案。

$$F(x) = \prod_{i=1}^n \frac{1}{1 - x^{v_i}}$$

$$F(x) = \exp \left(\sum_{i=1}^n -\ln(1 - x^{v_i}) \right)$$

$$F(x) = \exp \left(\sum_{i=1}^n \sum_{j \geq 0} \frac{x^{v_{ij}}}{j} \right)$$

- 调和级数求出 $\ln F(x)$ ，多项式 $\exp$ 即可，时间复杂度 $\Theta(n \log n)$ 。

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。
- 一个序列 a 的 EGF 定义为:

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。
- 一个序列 a 的 EGF 定义为:
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。
- 一个序列 a 的 EGF 定义为:
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 加法与 OGF 相同, 乘法在 a 序列上为二项加法卷积, 在 $\frac{a_n}{n!}$ 序列上为普通卷积。

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。
- 一个序列 a 的 EGF 定义为:
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 加法与 OGF 相同, 乘法在 a 序列上为二项加法卷积, 在 $\frac{a_n}{n!}$ 序列上为普通卷积。
- 设 $H(x) = F(x)G(x)$:

EGF 的定义

- EGF, 全称 exponential generating function, 指数生成函数。
- 一个序列 a 的 EGF 定义为:
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 加法与 OGF 相同, 乘法在 a 序列上为二项加法卷积, 在 $\frac{a_n}{n!}$ 序列上为普通卷积。
- 设 $H(x) = F(x)G(x)$:
-

$$\left[\frac{x^n}{n!} \right] H(x) = \sum_{i+j=n} \left[\frac{x^i}{i!} \right] F(x) \left[\frac{x^j}{j!} \right] G(x)$$

$$[x^n] H(x) = \sum_{i+j=n} \frac{n!}{i!j!} [x^i] F(x) [x^j] G(x)$$

$$[x^n] H(x) = \sum_{i+j=n} \binom{n}{i} [x^i] F(x) [x^j] G(j)$$

- 与 OGF 不同，EGF 一般用来处理有标号体系。

- 与 OGF 不同，EGF 一般用来处理有标号体系。
- 考虑若干小球，每个小球都有颜色。

- 与 OGF 不同，EGF 一般用来处理有标号体系。
- 考虑若干小球，每个小球都有颜色。
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 与 OGF 不同，EGF 一般用来处理有标号体系。
- 考虑若干小球，每个小球都有颜色。
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 将 F 看作一个小球的序列， $a_n \cdot n!$ 表示考虑小球的颜色，长度为 n 的不同小球序列有多少种。

- 与 OGF 不同, EGF 一般用来处理有标号体系。
- 考虑若干小球, 每个小球都有颜色。
-

$$F(x) = \sum_{n \geq 0} \frac{a_n x^n}{n!}$$

- 将 F 看作一个小球的序列, $a_n \cdot n!$ 表示考虑小球的颜色, 长度为 n 的不同小球序列有多少种。
- $F(x)G(x)$ 就相当于将两个颜色没有交集的小球序列合并。

- $F(x) = \sum_{n \geq 0} \frac{x^n}{n!} \rightarrow F(x) = e^x$
证明：泰勒展开即可。

- $F(x) = \sum_{n \geq 0} \frac{x^n}{n!} \rightarrow F(x) = e^x$
证明：泰勒展开即可。
- $F(x) = \sum_{n \geq 0} \frac{z^n x^n}{n!} \rightarrow F(x) = e^{zx}$
证明：将 zx 代入第一个式子即可。

- $F(x) = \sum_{n \geq 0} \frac{x^n}{n!} \rightarrow F(x) = e^x$

证明：泰勒展开即可。

- $F(x) = \sum_{n \geq 0} \frac{z^n x^n}{n!} \rightarrow F(x) = e^{zx}$

证明：将 zx 代入第一个式子即可。

- $F(x) = \sum_{n \geq 0} [n \equiv 0 \pmod{2}] x^n \rightarrow F(x) = \frac{e^x + e^{-x}}{2}$

- $F(x) = \sum_{n \geq 0} \frac{x^n}{n!} \rightarrow F(x) = e^x$

证明：泰勒展开即可。

- $F(x) = \sum_{n \geq 0} \frac{z^n x^n}{n!} \rightarrow F(x) = e^{zx}$

证明：将 zx 代入第一个式子即可。

- $F(x) = \sum_{n \geq 0} [n \equiv 0 \pmod{2}] x^n \rightarrow F(x) = \frac{e^x + e^{-x}}{2}$

- 拓展： $F(x) = \sum_{n \geq 0} [n \equiv 0 \pmod{p}] x^n \rightarrow F(x) = \frac{1}{p} \sum_{0 \leq i < p} e^{\omega_p^i x}$

其中 ω_n^m 即 n 次单位根的 m 次幂。

常用的 EGF

- $F(x) = \sum_{n \geq 0} \frac{x^n}{n!} \rightarrow F(x) = e^x$
证明：泰勒展开即可。
- $F(x) = \sum_{n \geq 0} \frac{z^n x^n}{n!} \rightarrow F(x) = e^{zx}$
证明：将 zx 代入第一个式子即可。
- $F(x) = \sum_{n \geq 0} [n \equiv 0 \pmod{2}] x^n \rightarrow F(x) = \frac{e^x + e^{-x}}{2}$
- 拓展： $F(x) = \sum_{n \geq 0} [n \equiv 0 \pmod{p}] x^n \rightarrow F(x) = \frac{1}{p} \sum_{0 \leq i < p} e^{\omega_p^i}$
其中 ω_n^m 即 n 次单位根的 m 次幂。
- $F(x) = \sum_{n \geq 0} a^n x^n \rightarrow F(x) = (1+x)^a$

- 群里有 k 个不同的复读机。为了庆祝平安夜的到来，在接下来的 n 秒内，它们每秒钟都会选出一位优秀的复读机进行复读。非常滑稽的是，一个复读机只有总共复读了 d 的倍数次才会感到快乐。问有多少种不同的安排方式使得所有的复读机都感到快乐，答案对质数 19491001 取模。
- $n \leq 10^9$
- $k \leq 5 \times 10^5$ 且 $d \in \{1, 2\}$ ，或 $k \leq 10^3$ 且 $d = 3$ 。

- $d = 1$ 时答案为 k^n 。

- $d = 1$ 时答案为 k^n 。
- $d = 2$ 时答案为 $[x^n] \left(\frac{e^x + e^{-x}}{2} \right)^k$ 。

- $d = 1$ 时答案为 k^n 。
- $d = 2$ 时答案为 $[x^n] \left(\frac{e^x + e^{-x}}{2} \right)^k$ 。
-

$$\begin{aligned}
 [x^n] \left(\frac{e^x + e^{-x}}{2} \right)^k &= 2^{-k} [x^n] \sum_{i=0}^k \binom{k}{i} e^{xi-x(k-i)} \\
 &= 2^{-k} \sum_{i=0}^k \binom{k}{i} (2i-k)^n
 \end{aligned}$$

- $d = 1$ 时答案为 k^n 。
- $d = 2$ 时答案为 $[x^n] \left(\frac{e^x + e^{-x}}{2} \right)^k$ 。
-

$$\begin{aligned}
 [x^n] \left(\frac{e^x + e^{-x}}{2} \right)^k &= 2^{-k} [x^n] \sum_{i=0}^k \binom{k}{i} e^{xi-x(k-i)} \\
 &= 2^{-k} \sum_{i=0}^k \binom{k}{i} (2i-k)^n
 \end{aligned}$$

- 直接计算时间复杂度 $\Theta(k \log n)$ 。

- $d = 3$ 时答案为 $[x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k$ 。

- $d = 3$ 时答案为 $[x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k$ 。

-

$$\begin{aligned}
 [x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k &= 3^{-k} [x^n] \sum_{a+b+c=k} \binom{k}{a, b, c} e^{ax} e^{\omega_3^1 bx} e^{\omega_3^2 cx} \\
 &= 3^{-k} \sum_{a+b+c=k} \binom{k}{a, b, c} (a + \omega_3^1 b + \omega_3^2 c)^n
 \end{aligned}$$

- $d = 3$ 时答案为 $[x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k$ 。

$$\begin{aligned}
 [x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k &= 3^{-k} [x^n] \sum_{a+b+c=k} \binom{k}{a, b, c} e^{ax} e^{\omega_3^1 bx} e^{\omega_3^2 cx} \\
 &= 3^{-k} \sum_{a+b+c=k} \binom{k}{a, b, c} (a + \omega_3^1 b + \omega_3^2 c)^n
 \end{aligned}$$

- 由于 3 在模 19491001 意义下有二次剩余，所以直接自己写个复数类计算即可。

- $d = 3$ 时答案为 $[x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k$ 。

-

$$\begin{aligned}
 [x^n] \left(\frac{e^x + e^{\omega_3^1 x} + e^{\omega_3^2 x}}{3} \right)^k &= 3^{-k} [x^n] \sum_{a+b+c=k} \binom{k}{a, b, c} e^{ax} e^{\omega_3^1 bx} e^{\omega_3^2 cx} \\
 &= 3^{-k} \sum_{a+b+c=k} \binom{k}{a, b, c} (a + \omega_3^1 b + \omega_3^2 c)^n
 \end{aligned}$$

- 由于 3 在模 19491001 意义下有二次剩余，所以直接自己写个复数类计算即可。
- 时间复杂度 $\Theta(k^2 \log n)$ 。

- 求 n 个点有标号无根树的数量，要求每个点度数的最大值为 m ，答案对 998244353 取模。
- $n, m \leq 5 \times 10^4$ 。

- 考虑 Prüfer 序列，相当于求长度为 $n - 2$ 的序列数量，使得序列中的每个数在 $[1, n]$ 范围内，并且 $1 \sim n$ 在序列中出现次数的最大值为 m 。

- 考虑 Prüfer 序列，相当于求长度为 $n - 2$ 的序列数量，使得序列中的每个数在 $[1, n]$ 范围内，并且 $1 \sim n$ 在序列中出现次数的最大值为 m 。
- 计算出最大值 $\leq m$ 的序列数量减去最大值 $\leq m - 1$ 的序列数量就是答案。

- 考虑 Prüfer 序列，相当于求长度为 $n - 2$ 的序列数量，使得序列中的每个数在 $[1, n]$ 范围内，并且 $1 \sim n$ 在序列中出现次数的最大值为 m 。
- 计算出最大值 $\leq m$ 的序列数量减去最大值 $\leq m - 1$ 的序列数量就是答案。
- 容易发现最大值 $\leq m$ 的序列数量为：

$$[x^{n-2}] \left(\sum_{0 \leq i \leq m} \frac{x^i}{i!} \right)^n$$

- 考虑 Prüfer 序列，相当于求长度为 $n - 2$ 的序列数量，使得序列中的每个数在 $[1, n]$ 范围内，并且 $1 \sim n$ 在序列中出现次数的最大值为 m 。
- 计算出最大值 $\leq m$ 的序列数量减去最大值 $\leq m - 1$ 的序列数量就是答案。
- 容易发现最大值 $\leq m$ 的序列数量为：

$$[x^{n-2}] \left(\sum_{0 \leq i \leq m} \frac{x^i}{i!} \right)^n$$

- 直接多项式快速幂即可，时间复杂度 $\Theta(n \log n)$ 或者 $\Theta(n \log^2 n)$ 。

- $\exp F(x)$ 的展开式: $\sum_{n \geq 0} \frac{F(x)^n}{n!}$ 。

- $\exp F(x)$ 的展开式: $\sum_{n \geq 0} \frac{F(x)^n}{n!}$ 。
- 若把 $F(x)$ 看作“单个元素”的 EGF, 则 $\exp F(x)$ 描述了这些元素组成的有标号集合。

- $\exp F(x)$ 的展开式： $\sum_{n \geq 0} \frac{F(x)^n}{n!}$ 。
- 若把 $F(x)$ 看作“单个元素”的 EGF，则 $\exp F(x)$ 描述了这些元素组成的有标号集合。
- 展开式的意义是：枚举有几个元素，多次卷积拼接，由于元素之间无序需要除以个数的阶乘。

exp 的组合意义

- $\exp F(x)$ 的展开式： $\sum_{n \geq 0} \frac{F(x)^n}{n!}$ 。
- 若把 $F(x)$ 看作“单个元素”的 EGF，则 $\exp F(x)$ 描述了这些元素组成的有标号集合。
- 展开式的意义是：枚举有几个元素，多次卷积拼接，由于元素之间无序需要除以个数的阶乘。
- 这能够给我们极大的帮助。对于一个集合计数的问题，我们只需要弄清单个元素的特征，就能 $\exp$ 而得到答案。

- 求 n 个点的简单有标号无向连通图数量，答案对 NTT 模数 1004535809 取模。
- $n \leq 1.3 \times 10^5$ 。

- 设 $F(x)$ 表示有标号无向连通图的 EGF, $G(x)$ 表示有标号无向一般图的 EGF。

- 设 $F(x)$ 表示有标号无向连通图的 EGF, $G(x)$ 表示有标号无向一般图的 EGF。
- 由于一般图是由若干个连通分量组成的, 所以 $G(x) = \exp F(x)$ 。

- 设 $F(x)$ 表示有标号无向连通图的 EGF, $G(x)$ 表示有标号无向一般图的 EGF。
- 由于一般图是由若干个连通分量组成的, 所以 $G(x) = \exp F(x)$ 。
- $F(x) = \ln G(x)$, 而 $G(x) = \sum_{n \geq 0} \frac{2^{n(n-1)/2} x^n}{n!}$ 。

- 设 $F(x)$ 表示有标号无向连通图的 EGF, $G(x)$ 表示有标号无向一般图的 EGF。
- 由于一般图是由若干个连通分量组成的, 所以 $G(x) = \exp F(x)$ 。
- $F(x) = \ln G(x)$, 而 $G(x) = \sum_{n \geq 0} \frac{2^{n(n-1)/2} x^n}{n!}$ 。
- 多项式 $\ln$ 即可, 时间复杂度 $\Theta(n \log n)$ 。

- PGF, 全称 probabilistic generating function, 概率生成函数。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。
- 对于一个随机变量 X , 约定 $X \in \mathbb{N}$ 。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。
- 对于一个随机变量 X , 约定 $X \in \mathbb{N}$ 。
- 其概率生成函数为 $F(z) = \sum_{n \geq 0} P(X = n)z^n$ 。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。
- 对于一个随机变量 X , 约定 $X \in \mathbb{N}$ 。
- 其概率生成函数为 $F(z) = \sum_{n \geq 0} P(X = n)z^n$ 。
- 显然有 $F(1) = \sum_{n \geq 0} P(X = n) = 1$ 。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。
- 对于一个随机变量 X , 约定 $X \in \mathbb{N}$ 。
- 其概率生成函数为 $F(z) = \sum_{n \geq 0} P(X = n)z^n$ 。
- 显然有 $F(1) = \sum_{n \geq 0} P(X = n) = 1$ 。
- $E(X) = \sum_{n \geq 0} P(X = n) \cdot X = F'(1)$ 。

- PGF, 全称 probabilistic generating function, 概率生成函数。
- 并不是用于组合对象（集合大小）计数，而是用于概率期望领域。
- 设 $P(A)$ 表示事件 A 发生的概率。
- 对于一个随机变量 X , 约定 $X \in \mathbb{N}$ 。
- 其概率生成函数为 $F(z) = \sum_{n \geq 0} P(X = n)z^n$ 。
- 显然有 $F(1) = \sum_{n \geq 0} P(X = n) = 1$ 。
- $E(X) = \sum_{n \geq 0} P(X = n) \cdot n = F'(1)$ 。
- 扩展一下能够得到 $E(X^k) = F^{(k)}(1)$, 即求 k 次导。

- 给定字符集大小 n 以及一个长度为 m 的字符串 s ，有一个初始为空的序列，每次在序列后加入一个随机字符（即每种字符有 $\frac{1}{n}$ 的概率选到），序列中有子串等于 s 时停止加入字符，求序列的期望长度。
- T 组数据，答案对 10^4 取模。
- $1 \leq n, m \leq 10^5, 1 \leq T \leq 50$ 。

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。
- 易得 $F(x) + G(x) = xG(x) + 1$ 。

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。
- 易得 $F(x) + G(x) = xG(x) + 1$ 。
- 我们考虑在一个未完成的串后面接上 s 使其强制结束，得到生成函数 $G(x)\left(\frac{x}{n}\right)^m$ 。

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。
- 易得 $F(x) + G(x) = xG(x) + 1$ 。
- 我们考虑在一个未完成的串后面接上 s 使其强制结束，得到生成函数 $G(x)\left(\frac{x}{n}\right)^m$ 。
- 考虑用 $F(x)$ 表示出同样的生成函数，由于一个未完成的串在接上 s 的时候可能还没接完就满足条件了，容易发现这种情况下接进去的一定是 s 的 border。

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。
- 易得 $F(x) + G(x) = xG(x) + 1$ 。
- 我们考虑在一个未完成的串后面接上 s 使其强制结束，得到生成函数 $G(x)\left(\frac{x}{n}\right)^m$ 。
- 考虑用 $F(x)$ 表示出同样的生成函数，由于一个未完成的串在接上 s 的时候可能还没接完就满足条件了，容易发现这种情况下接进去的一定是 s 的 border。
- 所以可以枚举满足条件后，多加进去的串长，得出
$$\sum_{1 \leq k \leq m, s[1,k]=s[m-k+1,m]} F(x)\left(\frac{x}{n}\right)^{k-1}。$$

- 建立生成函数 $F(x), G(x)$ ，其中 $[x^n]F(x)$ 表示恰好在第 n 次结束的概率， $[x^n]G(x)$ 表示在第 n 次还未结束的概率。
- 易得 $F(x) + G(x) = xG(x) + 1$ 。
- 我们考虑在一个未完成的串后面接上 s 使其强制结束，得到生成函数 $G(x)\left(\frac{x}{n}\right)^m$ 。
- 考虑用 $F(x)$ 表示出同样的生成函数，由于一个未完成的串在接上 s 的时候可能还没接完就满足条件了，容易发现这种情况下接进去的一定是 s 的 border。
- 所以可以枚举满足条件后，多加进去的串长，得出

$$\sum_{1 \leq k \leq m, s[1,k]=s[m-k+1,m]} F(x) \left(\frac{x}{n}\right)^{k-i}。$$

$$G(x) \left(\frac{x}{n}\right)^m = \sum_{1 \leq k \leq m, s[1,k]=s[m-k+1,m]} F(x) \left(\frac{x}{n}\right)^{k-i}$$

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $x = 1$ 代入得：

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $x = 1$ 代入得：

-

$$F'(1) = G(1)$$

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $x = 1$ 代入得：

-

$$F'(1) = G(1)$$

- 由于 $F'(1)$ 即为所求，所以只需求出 $G(1)$ 即可。

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $x = 1$ 代入得：

-

$$F'(1) = G(1)$$

- 由于 $F'(1)$ 即为所求，所以只需求出 $G(1)$ 即可。

-

$$G(1)\left(\frac{1}{n}\right)^m = \sum_{1 \leq k \leq m, s[1,k]=s[m-k+1,m]} F(1)\left(\frac{1}{n}\right)^{k-i}$$

- 将 $F(x) + G(x) = xG(x) + 1$ 两边求导得：

-

$$F'(x) + G'(x) = xG'(x) + G(x)$$

- 将 $x = 1$ 代入得：

-

$$F'(1) = G(1)$$

- 由于 $F'(1)$ 即为所求，所以只需要求出 $G(1)$ 即可。

-

$$G(1)\left(\frac{1}{n}\right)^m = \sum_{1 \leq k \leq m, s[1,k]=s[m-k+1,m]} F(1)\left(\frac{1}{n}\right)^{k-i}$$

- 因为 $F(1) = 1$ ，所以答案可以 $\Theta(m)$ 算出。

生成函数杂项

- 利用生成函数刻画出递推式，通过幂级数运算得出结果。

- 利用生成函数刻画出递推式，通过幂级数运算得出结果。
- 或者通过幂级数运算，将生成函数的系数用递推式表示。

- 在 $[1, n]$ 的区间里放 m 棵树，每棵树的高度为 k 。求有多少种放置树的方法，满足：
- 每个树都在整点上，且每个点最多只能放一棵树。
- 存在一种砍倒树的方案，使得树倒了之后不会越界，也不会有某个点被超过一棵树占据。你可以自由选择树向左倒（也就是占据区间 $[x - k, x]$ ）或者向右倒（也就是占据区间 $[x, x + k]$ ）。
- $1 \leq n, m, k \leq 3 \times 10^5$ 。

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。
- 考虑 dp，设 $f_{i,j}$ 表示当前考虑到第 i 棵树，且目前最靠右的被占据的位置为 j 。

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。
- 考虑 dp，设 $f_{i,j}$ 表示当前考虑到第 i 棵树，且目前最靠右的被占据的位置为 j 。
- 对于每一个状态 $f_{i,j}$ ，其贡献到的状态为：

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall 1 \leq i \leq k)$$

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall i \geq 1)$$

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。
- 考虑 dp，设 $f_{i,j}$ 表示当前考虑到第 i 棵树，且目前最靠右的被占据的位置为 j 。
- 对于每一个状态 $f_{i,j}$ ，其贡献到的状态为：

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall 1 \leq i \leq k)$$

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall i \geq 1)$$

- 初始状态 $f_{0,0} = 1$ 。

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。
- 考虑 dp，设 $f_{i,j}$ 表示当前考虑到第 i 棵树，且目前最靠右的被占据的位置为 j 。
- 对于每一个状态 $f_{i,j}$ ，其贡献到的状态为：

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall 1 \leq i \leq k)$$

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall i \geq 1)$$

- 初始状态 $f_{0,0} = 1$ 。
- 设 $F_i(x) = \sum_{j \geq 0} f_{i,j} x^j$ 。

- 对于一种方案，贪心地能往左倒就往左倒，否则往右倒，如果存在一种能砍倒树的方案，则这种方案合法。
- 考虑 dp，设 $f_{i,j}$ 表示当前考虑到第 i 棵树，且目前最靠右的被占据的位置为 j 。
- 对于每一个状态 $f_{i,j}$ ，其贡献到的状态为：

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall 1 \leq i \leq k)$$

$$f_{i,j} \rightarrow f_{i+1,j+k+i} (\forall i \geq 1)$$

- 初始状态 $f_{0,0} = 1$ 。
- 设 $F_i(x) = \sum_{j \geq 0} f_{i,j} x^j$ 。
-

$$F_{i+1}(x) = F_i(x) \left(\frac{x^{k+1} - x^{2k+1}}{1-x} + \frac{x^{k+1}}{1-x} \right) = F_i(x) \frac{x^{k+1}(2-x^k)}{1-x}$$

-

$$F_m(x) = \frac{x^{(k+1)m}(2 - x^k)^m}{(1 - x)^m}$$

- $$F_m(x) = \frac{x^{(k+1)m}(2 - x^k)^m}{(1 - x)^m}$$
- 需要计算的是 $\sum_{i=0}^n [x^i]F_m(x)$ ，转化为 $[x^n]\frac{F_m(x)}{1 - x}$ 。

•

$$F_m(x) = \frac{x^{(k+1)m}(2-x^k)^m}{(1-x)^m}$$

• 需要计算的是 $\sum_{i=0}^n [x^i] F_m(x)$, 转化为 $[x^n] \frac{F_m(x)}{1-x}$ 。

•

$$\begin{aligned}
 & [x^n] \frac{x^{(k+1)m}(2-x^k)^m}{(1-x)^{m+1}} \\
 &= [x^{n-(k+1)m}] \sum_{i=0}^m \binom{m}{i} 2^i (-1)^{m-i} x^{k(m-i)} \left(\frac{1}{1-x} \right)^{m+1}
 \end{aligned}$$

-

$$F_m(x) = \frac{x^{(k+1)m}(2-x^k)^m}{(1-x)^m}$$

- 需要计算的是 $\sum_{i=0}^n [x^i] F_m(x)$, 转化为 $[x^n] \frac{F_m(x)}{1-x}$ 。

-

$$\begin{aligned} & [x^n] \frac{x^{(k+1)m}(2-x^k)^m}{(1-x)^{m+1}} \\ &= [x^{n-(k+1)m}] \sum_{i=0}^m \binom{m}{i} 2^i (-1)^{m-i} x^{k(m-i)} \left(\frac{1}{1-x} \right)^{m+1} \end{aligned}$$

- 先将二项式定理部分的幂级数算出来, 然后 $\left(\frac{1}{1-x} \right)^{m+1}$ 相当于做 $m+1$ 次前缀和, 由于只需要算一项, 直接卷积即可, 时间复杂度线性。

- 你有两个变量 V, W ，初始时 $V = 0, W = 1$ 。
- 接下来有 $n + m$ 场比赛，其中你赢了 n 场输了 m 场。
- 你赢一场比赛，则 V 变为 $V + 2$ ，然后 W 变为 $W \times V$ 。你输一场比赛，则 V 变为 $V - 1$ 。
- 对于所有可能的输赢情况，求出 W 最终取值的和。
- $n \leq 5 \times 10^4, m \leq 10^5$ ，时限 12s。

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 w 的和。

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 w 的和。
- 边界情况 $f_{0,0} = 1$ 。

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。
-

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。
-

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。

-

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

-

$$F_x(z) = zF_{x-1}(z) + 2xF_{x-1}(z) - 3zF'_{x-1}(z)$$

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。

-

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

-

$$F_x(z) = zF_{x-1}(z) + 2xF_{x-1}(z) - 3zF'_{x-1}(z)$$

- 左右同乘 $e^{-z/3}$ 。

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。

-

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

-

$$F_x(z) = zF_{x-1}(z) + 2xF_{x-1}(z) - 3zF'_{x-1}(z)$$

- 左右同乘 $e^{-z/3}$ 。

-

$$F_x(z)e^{-z/3} = zF_{x-1}(z)e^{-z/3} + 2xF_{x-1}(z)e^{-z/3} - 3zF'_{x-1}(z)e^{-z/3}$$

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

$$F_x(z) = zF_{x-1}(z) + 2xF_{x-1}(z) - 3zF'_{x-1}(z)$$

- 左右同乘 $e^{-z/3}$ 。

$$F_x(z)e^{-z/3} = zF_{x-1}(z)e^{-z/3} + 2xF_{x-1}(z)e^{-z/3} - 3zF'_{x-1}(z)e^{-z/3}$$

- 由于 $(F_x(z)e^{-z/3})' = F'_x(z)e^{-z/3} - 1/3F_x(z)e^{-z/3}$ ，所以：

- 设 $f_{x,y}$ 表示赢了 x 场比赛，输了 y 场，所有可能情况 W 的和。
- 边界情况 $f_{0,0} = 1$ 。

$$f_{x,y} = f_{x-1,y-1} + f_{x-1,y}(2x - 3y)$$

- 考虑设 $F_x(z) = \sum_{y \geq 0} f_{x,y} z^y$ 。

$$F_x(z) = zF_{x-1}(z) + 2xF_{x-1}(z) - 3zF'_{x-1}(z)$$

- 左右同乘 $e^{-z/3}$ 。

$$F_x(z)e^{-z/3} = zF_{x-1}(z)e^{-z/3} + 2xF_{x-1}(z)e^{-z/3} - 3zF'_{x-1}(z)e^{-z/3}$$

- 由于 $(F_x(z)e^{-z/3})' = F'_x(z)e^{-z/3} - 1/3F_x(z)e^{-z/3}$ ，所以：

$$F_x(z)e^{-z/3} = 2xF_{x-1}(z)e^{-z/3} - 3z(F_{x-1}(z)e^{-z/3})'$$

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

-

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

-

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

-

$$[z^y]H_x(z) = 2x[z^y]H_{x-1}(z) - 3y[z^y]H_{x-1}(z)$$

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

-

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

-

$$[z^y]H_x(z) = 2x[z^y]H_{x-1}(z) - 3y[z^y]H_{x-1}(z)$$

-

$$[z^y]H_x(z) = (2x - 3y)H_{x-1}(z)$$

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

-

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

-

$$[z^y]H_x(z) = 2x[z^y]H_{x-1}(z) - 3y[z^y]H_{x-1}(z)$$

-

$$[z^y]H_x(z) = (2x - 3y)H_{x-1}(z)$$

-

$$[z^y]H_x(z) = \prod_{i=1}^x (2i - 3y)[z^y]H_0(z)$$

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

-

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

-

$$[z^y]H_x(z) = 2x[z^y]H_{x-1}(z) - 3y[z^y]H_{x-1}(z)$$

-

$$[z^y]H_x(z) = (2x - 3y)H_{x-1}(z)$$

-

$$[z^y]H_x(z) = \prod_{i=1}^x (2i - 3y)[z^y]H_0(z)$$

- 其中 $H_0(z) = e^{-z/3}$ 。

- 设 $H_x(z) = F_x(z)e^{-z/3}$, 则:

$$H_x(z) = 2xH_{x-1}(z) - 3zH'_{x-1}(z)$$

$$[z^y]H_x(z) = 2x[z^y]H_{x-1}(z) - 3y[z^y]H_{x-1}(z)$$

$$[z^y]H_x(z) = (2x - 3y)H_{x-1}(z)$$

$$[z^y]H_x(z) = \prod_{i=1}^x (2i - 3y)[z^y]H_0(z)$$

- 其中 $H_0(z) = e^{-z/3}$ 。
- 容易 $\Theta(n + m)$ 算出 $H_{n+m}(z)$, 再卷 $e^{z/3}$ 即可恢复到 $F_{n+m}(z)$, 由于只需要计算 F 的一项, 时间复杂度 $\Theta(n + m)$ 。

- 给定 n 和一个 k 次多项式 $f(x)$ ，求 $f^n(x)$ 的各项系数。
- $1 \leq n \leq 10^5, 0 \leq k \leq 9$ 。

- 设 $g(x) = f^n(x)$ ，求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。

- 设 $g(x) = f^n(x)$ ，求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。
- 两边同乘 $f(x)$ ，可得 $g'(x)f(x) = nf'(x)g(x)$ 。

- 设 $g(x) = f^n(x)$, 求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。
- 两边同乘 $f(x)$, 可得 $g'(x)f(x) = nf'(x)g(x)$ 。
- 设 $f_n = [x^n]f(x)$, $g_n = [x^n]g(x)$, 当 $n < 0$ 时 $f_n = g_n = 0$ 。

- 设 $g(x) = f^n(x)$ ，求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。
- 两边同乘 $f(x)$ ，可得 $g'(x)f(x) = nf'(x)g(x)$ 。
- 设 $f_n = [x^n]f(x)$, $g_n = [x^n]g(x)$ ，当 $n < 0$ 时 $f_n = g_n = 0$ 。
- 考虑 x^m 处的系数，可以得到：

$$\sum_{i=0}^k f_i g_{m-i+1} (m-i+1) = \sum_{i=0}^k n g_{m-i} f_{i+1} (i+1)$$

- 设 $g(x) = f^n(x)$, 求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。
- 两边同乘 $f(x)$, 可得 $g'(x)f(x) = nf'(x)g(x)$ 。
- 设 $f_n = [x^n]f(x)$, $g_n = [x^n]g(x)$, 当 $n < 0$ 时 $f_n = g_n = 0$ 。
- 考虑 x^m 处的系数, 可以得到:

$$\sum_{i=0}^k f_i g_{m-i+1} (m-i+1) = \sum_{i=0}^k n g_{m-i} f_{i+1} (i+1)$$

•

$$(m+1)g_{m+1}f_0 = \sum_{i=0}^k n g_{m-i} f_{i+1} (i+1) - \sum_{i=1}^k f_i g_{m-i+1} (m-i+1)$$

- 设 $g(x) = f^n(x)$, 求导可得 $g'(x) = f'(x)nf^{n-1}(x)$ 。
- 两边同乘 $f(x)$, 可得 $g'(x)f(x) = nf'(x)g(x)$ 。
- 设 $f_n = [x^n]f(x)$, $g_n = [x^n]g(x)$, 当 $n < 0$ 时 $f_n = g_n = 0$ 。
- 考虑 x^m 处的系数, 可以得到:

$$\sum_{i=0}^k f_i g_{m-i+1} (m-i+1) = \sum_{i=0}^k n g_{m-i} f_{i+1} (i+1)$$

•

$$(m+1)g_{m+1}f_0 = \sum_{i=0}^k n g_{m-i} f_{i+1} (i+1) - \sum_{i=1}^k f_i g_{m-i+1} (m-i+1)$$

- $\Theta(nk^2)$ 递推即可。

- 设欧拉函数 $\phi(x) = \prod_{k \geq 1} (1 - x^k)$ 。

- 设欧拉函数 $\phi(x) = \prod_{k \geq 1} (1 - x^k)$ 。
- 则 $\phi(x) = 1 + \sum_{k \geq 1} (-1)^k x^{k(3k \pm 1)}$ 。

- 设欧拉函数 $\phi(x) = \prod_{k \geq 1} (1 - x^k)$ 。
- 则 $\phi(x) = 1 + \sum_{k \geq 1} (-1)^k x^{k(3k \pm 1)}$ 。
- 具体的挂个链接：[五边形数定理](#)。

- 顺带着提一嘴……

- 顺带着提一嘴……
- 讲几道生成函数解决 Prüfer 序列计数的例题。

- 有 N 个点，每个点有 d_i 个孔，每次可以选择两个不同点，连接两个未被连接过的孔，有多少种方案使得最后形成一棵树。
- $2 \leq N \leq 2 \times 10^5, 1 \leq d_i < 998244353$ 。

- 挂个链接：[\[ARC106F\] Figures](#)。

- 在一个 s 个点的图中，存在 $s - n$ 条边，使图中形成了 n 个连通块，第 i 个连通块中有 a_i 个点。
- 现在我们需要再连接 $n - 1$ 条边，使该图变成一棵树。对一种连边方案，设原图中第 i 个连通块连出了 d_i 条边，那么这棵树 T 的价值为：

$$\text{val}(T) = \left(\prod_{i=1}^n d_i^m \right) \left(\sum_{i=1}^n d_i^m \right)$$

- 你的任务是求出所有可能的生成树的价值之和，对 998244353 取模。
- $n \leq 3 \times 10^4, m \leq 30, 1 \leq a_i < 998244353$ 。

- 将生成树转 Prüfer 序列，设第 i 个连通块在 Prüfer 序列中出现了 d_i 次。

- 将生成树转 Prüfer 序列，设第 i 个连通块在 Prüfer 序列中出现了 d_i 次。
- 答案即为：

- 将生成树转 Prüfer 序列，设第 i 个连通块在 Prüfer 序列中出现了 d_i 次。
- 答案即为：
-

$$\sum_{d_1, \dots, d_n \geq 0, \sum d_i = n-2} \binom{n-2}{d_1, \dots, d_n} \left(\prod_{i=1}^n a_i^{d_i+1} \right) \left(\prod_{i=1}^n (d_i+1)^m \right) \left(\sum_{i=1}^n (d_i+1)^m \right)$$

- 将生成树转 Prüfer 序列，设第 i 个连通块在 Prüfer 序列中出现了 d_i 次。
- 答案即为：

$$\sum_{d_1, \dots, d_n \geq 0, \sum d_i = n-2} \binom{n-2}{d_1, \dots, d_n} \left(\prod_{i=1}^n a_i^{d_i+1} \right) \left(\prod_{i=1}^n (d_i+1)^m \right) \left(\sum_{i=1}^n (d_i+1)^m \right)$$

$$= \prod_{i=1}^n a_i \sum_{d_1, \dots, d_n \geq 0, \sum d_i = n-2} \binom{n-2}{d_1, \dots, d_n} \sum_{i=1}^n (d_i+1)^{2m} a_i^{d_i} \prod_{1 \leq j \leq n, j \neq i} (d_j+1)^m a_j^{d_j}$$

- 设 $A(x) = \sum_{k=0} (k+1)^m \frac{x^k}{k!}$, $B(x) = \sum_{k=0} (k+1)^{2m} \frac{x^k}{k!}$ 。

- 设 $A(x) = \sum_{k=0} (k+1)^m \frac{x^k}{k!}$, $B(x) = \sum_{k=0} (k+1)^{2m} \frac{x^k}{k!}$ 。
- 则答案即为：

- 设 $A(x) = \sum_{k=0} (k+1)^m \frac{x^k}{k!}$, $B(x) = \sum_{k=0} (k+1)^{2m} \frac{x^k}{k!}$.

- 则答案即为：

-

$$\prod_{i=1}^n a_i (n-2)! \left[\frac{x^{n-2}}{(n-2)!} \right] \sum_{i=1}^n A(a_i x) \prod_{j \neq i} B(a_j x)$$

- 设 $A(x) = \sum_{k=0} (k+1)^m \frac{x^k}{k!}$, $B(x) = \sum_{k=0} (k+1)^{2m} \frac{x^k}{k!}$ 。

- 则答案即为：

- $$\prod_{i=1}^n a_i (n-2)! \left[\frac{x^{n-2}}{(n-2)!} \right] \sum_{i=1}^n A(a_i x) \prod_{j \neq i} B(a_j x)$$

- $$= \prod_{i=1}^n a_i (n-2)! \left[\frac{x^{n-2}}{(n-2)!} \right] \sum_{i=1}^n \frac{A(a_i x)}{B(a_i x)} \exp \left(\sum_{j=1}^n \ln B(a_j x) \right)$$

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。
- 问题转化为求 $\sum_{i=1}^n C(a_i x)$ 和 $\sum_{i=1}^n D(a_i x)$ 。

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。
- 问题转化为求 $\sum_{i=1}^n C(a_i x)$ 和 $\sum_{i=1}^n D(a_i x)$ 。
- 发现只需要对 $t \in [0, n]$, 求出 $\sum_{i=1}^n a_i^t$ 。

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。
- 问题转化为求 $\sum_{i=1}^n C(a_i x)$ 和 $\sum_{i=1}^n D(a_i x)$ 。
- 发现只需要对 $t \in [0, n]$, 求出 $\sum_{i=1}^n a_i^t$ 。
- 注意到这相当于求 $[x^t] \sum_{i=1}^n \frac{1}{1 - a_i x}$ 。

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。
- 问题转化为求 $\sum_{i=1}^n C(a_i x)$ 和 $\sum_{i=1}^n D(a_i x)$ 。
- 发现只需要对 $t \in [0, n]$, 求出 $\sum_{i=1}^n a_i^t$ 。
- 注意到这相当于求 $[x^t] \sum_{i=1}^n \frac{1}{1 - a_i x}$ 。
- 又注意到 $\sum_{i=1}^n \frac{1}{1 - a_i x} = \frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)}$

- 设 $C(x) = \frac{A(x)}{B(x)}$, $D(x) = \ln B(x)$ 。
- $C(x)$ 和 $D(x)$ 分别可以通过多项式求逆与多项式 $\ln$ 在 $\Theta(n \log n)$ 的复杂度内求出。
- 问题转化为求 $\sum_{i=1}^n C(a_i x)$ 和 $\sum_{i=1}^n D(a_i x)$ 。
- 发现只需要对 $t \in [0, n]$, 求出 $\sum_{i=1}^n a_i^t$ 。
- 注意到这相当于求 $[x^t] \sum_{i=1}^n \frac{1}{1 - a_i x}$ 。
- 又注意到 $\sum_{i=1}^n \frac{1}{1 - a_i x} = \frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)}$
- 分子分母可以通过分治 NTT 一起求出, 时间复杂度 $\Theta(n \log^2 n)$ 。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。
- 设 $\frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)} = \frac{P(x)}{Q(x)}$ 。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。
- 设 $\frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)} = \frac{P(x)}{Q(x)}$ 。
- 首先分治 NTT 求出 $Q(x)$ ，考虑通过 $Q(x)$ 求出 $P(x)$ 。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。
- 设 $\frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)} = \frac{P(x)}{Q(x)}$ 。
- 首先分治 NTT 求出 $Q(x)$ ，考虑通过 $Q(x)$ 求出 $P(x)$ 。
- 设 $P_1(x) = \sum_{i=1}^n \prod_{j \neq i} (x - a_j)$, $Q_1(x) = \prod_{i=1}^n (x - a_i)$ 。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。
- 设 $\frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)} = \frac{P(x)}{Q(x)}$ 。
- 首先分治 NTT 求出 $Q(x)$ ，考虑通过 $Q(x)$ 求出 $P(x)$ 。
- 设 $P_1(x) = \sum_{i=1}^n \prod_{j \neq i} (x - a_j)$, $Q_1(x) = \prod_{i=1}^n (x - a_i)$ 。
- 容易发现 $P_1(x)$, $Q_1(x)$ 分别是 $P(x)$, $Q(x)$ 翻转了一下系数。

- 已经可以通过了，但在去年磁暴课件中还有常数更小的做法。
- 设 $\frac{\sum_{i=1}^n \prod_{j \neq i} (1 - a_j x)}{\prod_{i=1}^n (1 - a_i x)} = \frac{P(x)}{Q(x)}$ 。
- 首先分治 NTT 求出 $Q(x)$ ，考虑通过 $Q(x)$ 求出 $P(x)$ 。
- 设 $P_1(x) = \sum_{i=1}^n \prod_{j \neq i} (x - a_j)$, $Q_1(x) = \prod_{i=1}^n (x - a_i)$ 。
- 容易发现 $P_1(x)$, $Q_1(x)$ 分别是 $P(x)$, $Q(x)$ 翻转了一下系数。
- 又注意到 $P_1(x) = Q_1'(x)$ ，所以可以通过 $Q_1(x)$ 在 $\Theta(n)$ 复杂度内得到 $P_1(x)$ 。

生成函数杂题

- 生活在 998244353 号小宇宙的艾和兰收到了归零者的讯息，决定响应回归运动。他们要把大部分的物质归还给大宇宙，只留下极少的物质用于在新宇宙重建自己的文明。
- 艾和兰的文明总共有 n 个关键信息，编号为 $1, 2, \dots, n$ 。他们需要保留的信息是这些关键信息的一个子集 S 。对于一个编号为 x 的信息，只要 S 中一个子集的编号和等于 x ，那么他们设计的漂流瓶就可以在新宇宙将 x 还原出来。
- 艾和兰不禁想要思考，他们有多少种选择子集 S 的方案，使得关键信息 $1, 2, \dots, n$ 均能被还原？艾和兰自然是只用 1 微秒就算出了方案数的精确数值，现在他们想让你帮忙验算。由于方案数可能很大，你只需要输出方案数对 M 取模的结果。
- $1 \leq n \leq 5 \times 10^5, 2 \leq M \leq 1.01 \times 10^9$ 。

- 仍然是上面那个链接：五边形数定理。

- 给定一个长度为 n 的 01 数组 s 。
- 初始你有一个长度为 n 的数组 a ，其中每个位置上的数都是 0。
- 对于 $1, \dots, n$ 的每个位置 i ，有一个权值 w_i ，每次操作会以正比于 w_i 的概率选出一个位置 i ，并且将 a_i 异或上 1。
- 一旦 $a = s$ ，你就会停止操作，求期望操作次数，输出答案对 998244353 取模后的结果。
- $1 \leq n \leq 100, w_i \geq 1, \sum_{i=1}^n w_i \leq 5 \times 10^4$ 。

- 首先算出每个位置被选到的概率 $p_i = \frac{w_i}{\sum_{j=1}^n w_j}$ 。
- 由于操作之间是有顺序的，考虑 EGF。
- 设 $G(x) = \sum_{i=0} g_i \frac{x^i}{i!}$, $H(x) = \sum_{i=0} h_i \frac{x^i}{i!}$, g_i 表示从状态 s 开始操作 i 次后回到状态 s 的概率， h_i 表示操作 i 次后到达状态 s 的概率。

-

$$G(x) = \prod_{i=1}^n \frac{e^{p_i x} + e^{-p_i x}}{2}$$

$$H(x) = \prod_{i=1}^n [s_i = 0] \frac{e^{p_i x} + e^{-p_i x}}{2} + [s_i = 1] \frac{e^{p_i x} - e^{-p_i x}}{2}$$

- 设 $G(x) = \sum_i c_i e^{ix}$, $H(x) = \sum_i d_i e^{ix}$, 其中 c_i 和 d_i 可以用一个 $\Theta(n \sum w)$ 的背包算出。

- 设 $G(x) = \sum_i c_i e^{ix}$, $H(x) = \sum_i d_i e^{ix}$, 其中 c_i 和 d_i 可以用一个 $\Theta(n \sum w)$ 的背包算出。
- 考虑设 $P(x) = \sum_i \frac{c_i}{1 - ix}$, $Q(x) = \sum_i \frac{d_i}{1 - ix}$, 即 G, H 的 OGF 版本。

- 设 $G(x) = \sum_i c_i e^{ix}$, $H(x) = \sum_i d_i e^{ix}$, 其中 c_i 和 d_i 可以用一个 $\Theta(n \sum w)$ 的背包算出。
- 考虑设 $P(x) = \sum_i \frac{c_i}{1 - ix}$, $Q(x) = \sum_i \frac{d_i}{1 - ix}$, 即 G, H 的 OGF 版本。
- 设 $F(x) = \sum_{i=0}^n f_i x^i$, 其中 f_i 表示操作 i 次后第一次到达状态 s 的概率。易得 $F(x)P(x) = Q(x)$ 。

- 设 $G(x) = \sum_i c_i e^{ix}$, $H(x) = \sum_i d_i e^{ix}$, 其中 c_i 和 d_i 可以用一个 $\Theta(n \sum w)$ 的背包算出。
- 考虑设 $P(x) = \sum_i \frac{c_i}{1 - ix}$, $Q(x) = \sum_i \frac{d_i}{1 - ix}$, 即 G, H 的 OGF 版本。
- 设 $F(x) = \sum_{i=0}^n f_i x^i$, 其中 f_i 表示操作 i 次后第一次到达状态 s 的概率。易得 $F(x)P(x) = Q(x)$ 。
- 而 $F'(1)$ 就是答案。

- 设 $G(x) = \sum_i c_i e^{ix}$, $H(x) = \sum_i d_i e^{ix}$, 其中 c_i 和 d_i 可以用一个 $\Theta(n \sum w)$ 的背包算出。
- 考虑设 $P(x) = \sum_i \frac{c_i}{1 - ix}$, $Q(x) = \sum_i \frac{d_i}{1 - ix}$, 即 G, H 的 OGF 版本。
- 设 $F(x) = \sum_{i=0}^n f_i x^i$, 其中 f_i 表示操作 i 次后第一次到达状态 s 的概率。易得 $F(x)P(x) = Q(x)$ 。
- 而 $F'(1)$ 就是答案。
-

$$F'(1) = \left(\frac{Q(1)}{P(1)} \right)' = \frac{Q'(1)P(1) - Q(1)P'(1)}{P^2(1)}$$

- 但是计算的时候会发现当 $i = 1$ 时, $\frac{c_i}{1 - iX}$ (或 $\frac{d_i}{1 - iX}$) 会不收敛。

- 但是计算的时候会发现当 $i = 1$ 时, $\frac{c_i}{1 - ix}$ (或 $\frac{d_i}{1 - ix}$) 会不收敛。
- 由于答案一定收敛, 所以我们可以对这个分式做一些变换, 将 $P(x)$ 与 $Q(x)$ 同乘 $(1 - x)$ 。

- 但是计算的时候会发现当 $i = 1$ 时, $\frac{c_i}{1 - ix}$ (或 $\frac{d_i}{1 - ix}$) 会不收敛。
- 由于答案一定收敛, 所以我们可以对这个分式做一些变换, 将 $P(x)$ 与 $Q(x)$ 同乘 $(1 - x)$ 。
-

$$P(x) = c_1 + \sum_{i \neq 1} \frac{c_i(1 - x)}{1 - ix} \Rightarrow P(1) = c_1$$
$$Q(1) = d_1$$

- 但是计算的时候会发现当 $i = 1$ 时, $\frac{c_i}{1-ix}$ (或 $\frac{d_i}{1-ix}$) 会不收敛。
- 由于答案一定收敛, 所以我们可以对这个分式做一些变换, 将 $P(x)$ 与 $Q(x)$ 同乘 $(1-x)$ 。

$$P(x) = c_1 + \sum_{i \neq 1} \frac{c_i(1-x)}{1-ix} \Rightarrow P(1) = c_1$$

$$Q(1) = d_1$$

$$P'(x) = \sum_i c_i \frac{i(1-x) - (1-ix)}{(1-ix)^2}$$

$$P'(1) = \sum_{i \neq 1} \frac{c_i(i-1)}{(1-i)^2} = \sum_{i \neq 1} \frac{c_i}{i-1}$$

$$Q'(1) = \sum_{i \neq 1} \frac{d_i}{i-1}$$

- 但是计算的时候会发现当 $i = 1$ 时, $\frac{c_i}{1-ix}$ (或 $\frac{d_i}{1-ix}$) 会不收敛。
- 由于答案一定收敛, 所以我们可以对这个分式做一些变换, 将 $P(x)$ 与 $Q(x)$ 同乘 $(1-x)$ 。

$$P(x) = c_1 + \sum_{i \neq 1} \frac{c_i(1-x)}{1-ix} \Rightarrow P(1) = c_1$$

$$Q(1) = d_1$$

$$P'(x) = \sum_i c_i \frac{i(1-x) - (1-ix)}{(1-ix)^2}$$

$$P'(1) = \sum_{i \neq 1} \frac{c_i(i-1)}{(1-i)^2} = \sum_{i \neq 1} \frac{c_i}{i-1}$$

$$Q'(1) = \sum_{i \neq 1} \frac{d_i}{i-1}$$

- 现在就可以计算答案了, 时间复杂度 $\Theta(n \sum w)$ 。

符号化方法入门

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。
- 定义该组合类的生成函数（OGF）为：

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。
- 定义该组合类的生成函数（OGF）为：
-

$$A(z) = \sum_{a \in \mathcal{A}} z^{|a|} = \sum_{i=0} [z^i] A(z) z^i$$

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。
- 定义该组合类的生成函数（OGF）为：
-

$$A(z) = \sum_{a \in \mathcal{A}} z^{|a|} = \sum_{i=0} [z^i] A(z) z^i$$

- 生成函数是幂表示的和，故也是形式幂级数。

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。
- 定义该组合类的生成函数（OGF）为：
-

$$A(z) = \sum_{a \in \mathcal{A}} z^{|a|} = \sum_{i=0} [z^i] A(z) z^i$$

- 生成函数是幂表示的和，故也是形式幂级数。
- 由定义不难验证，幂表示的和代表单元素（集合）的并，生成函数的和表示集合的并。

组合结构导出生成函数

- 对于组合类 $\mathcal{A}$ 和组合对象 $a \in \mathcal{A}$ ，将 a 用代数对象 z 的幂表示为 $z^{|a|}$ ，称为幂表示。
- 在幂表示中，我们略去了组合对象的其他信息，只保留了“大小”这一个信息。
- 这里 z 只是一个服从乘法运算律的符号。
- 定义该组合类的生成函数（OGF）为：

$$A(z) = \sum_{a \in \mathcal{A}} z^{|a|} = \sum_{i=0} [z^i] A(z) z^i$$

- 生成函数是幂表示的和，故也是形式幂级数。
- 由定义不难验证，幂表示的和代表单元素（集合）的并，生成函数的和表示集合的并。
- 对于组合对象 a, b 的组合 (a, b) ，有 $z^{|(a,b)|} = z^{|a|+|b|} = z^{|a|} \times z^{|b|}$ 。于是，组合对象的组合对应幂表示的乘积。

- 利用乘法运算律

$(a_1 + a_2 + \cdots) \times (b_1 + b_2 + \cdots) = a_1b_1 + a_1b_2 + a_2b_1 + a_2b_2 + \cdots$ 可以验证，生成函数的乘积表示组合类的笛卡尔积。

组合结构导出生成函数

- 利用乘法运算律

$(a_1 + a_2 + \cdots) \times (b_1 + b_2 + \cdots) = a_1b_1 + a_1b_2 + a_2b_1 + a_2b_2 + \cdots$ 可以验证，生成函数的乘积表示组合类的笛卡尔积。

-

$$\mathcal{C} = \mathcal{A} + \mathcal{B} \Rightarrow C(z) = A(z) + B(z)$$

$$\mathcal{C} = \mathcal{A} \times \mathcal{B} \Rightarrow C(z) = A(z)B(z)$$

组合结构导出生成函数

- 利用乘法运算律

$(a_1 + a_2 + \cdots) \times (b_1 + b_2 + \cdots) = a_1b_1 + a_1b_2 + a_2b_1 + a_2b_2 + \cdots$ 可以验证，生成函数的乘积表示组合类的笛卡尔积。

-

$$\mathcal{C} = \mathcal{A} + \mathcal{B} \Rightarrow C(z) = A(z) + B(z)$$

$$\mathcal{C} = \mathcal{A} \times \mathcal{B} \Rightarrow C(z) = A(z)B(z)$$

- 综上，生成函数的基本运算和组合类的“集合并”，“笛卡尔积”是同构的。我们可以将组合类的构造翻译为生成函数的运算。

- 记 $\mathcal{E} = \{\epsilon\}$ 是只由一个大小为 0 的元素（空）构成的组合类。有 $E(z) = 1$ 。

- 记 $\mathcal{E} = \{\epsilon\}$ 是只由一个大小为 0 的元素（空）构成的组合类。有 $E(z) = 1$ 。
- 对于任意组合类 $\mathcal{A}$ ，规定 $\mathcal{A}^0 = \mathcal{E}$ 。但注意 $\mathcal{E}$ 并不能直接视作笛卡尔积的单位元。

一些基本的组合类

- 记 $\mathcal{E} = \{\epsilon\}$ 是只由一个大小为 0 的元素（空）构成的组合类。有 $E(z) = 1$ 。
- 对于任意组合类 $\mathcal{A}$ ，规定 $\mathcal{A}^0 = \mathcal{E}$ 。但注意 $\mathcal{E}$ 并不能直接视作笛卡尔积的单位元。
- 记 $\mathcal{Z}$ 是只由一个大小为 1 的元素构成的组合类。有 $Z(z) = z$ 。

- 设 01 串的组合类为 $\mathcal{S}$ 。

- 设 01 串的组合类为 $\mathcal{S}$ 。
-

$$\mathcal{S} = \mathcal{E} + 0 + 1 + 00 + 01 + 10 + 11 + \cdots$$

- 设 01 串的组合类为 $\mathcal{S}$ 。
-

$$\mathcal{S} = \mathcal{E} + 0 + 1 + 00 + 01 + 10 + 11 + \cdots$$

- 对于某个 01 串，要么为空，要么是 0,1 接上另一个 01 串。

- 设 01 串的组合类为 S 。

-

$$S = \mathcal{E} + 0 + 1 + 00 + 01 + 10 + 11 + \cdots$$

- 对于某个 01 串，要么为空，要么是 0,1 接上另一个 01 串。

-

$$S = \mathcal{E} + S \times (0 + 1)$$

01 串的组合类

- 设 01 串的组合类为 $\mathcal{S}$ 。

-

$$\mathcal{S} = \mathcal{E} + \emptyset + 1 + \emptyset\emptyset + \emptyset 1 + 1\emptyset + 11 + \cdots$$

- 对于某个 01 串，要么为空，要么是 0,1 接上另一个 01 串。

-

$$\mathcal{S} = \mathcal{E} + \mathcal{S} \times (\emptyset + 1)$$

- 翻译成生成函数则有 $S(z) = 1 + (z + z)S(z)$ ，可得 $S(z) = \frac{1}{1 - 2z}$ 。

01 串的组合类

- 设 01 串的组合类为 $\mathcal{S}$ 。

-

$$\mathcal{S} = \mathcal{E} + \emptyset + 1 + \emptyset\emptyset + \emptyset 1 + 1\emptyset + 11 + \dots$$

- 对于某个 01 串，要么为空，要么是 0,1 接上另一个 01 串。

-

$$\mathcal{S} = \mathcal{E} + \mathcal{S} \times (\emptyset + 1)$$

- 翻译成生成函数则有 $S(z) = 1 + (z + z)S(z)$ ，可得 $S(z) = \frac{1}{1 - 2z}$ 。
- 所以 $[z^n]S(z) = 2^n$ ，这与其组合意义相符。

- 卡特兰数有个广为人知的组合意义， C_n 表示 n 个点二叉树的个数。

- 卡特兰数有个广为人知的组合意义， C_n 表示 n 个点二叉树的个数。
- 设卡特兰数的组合类为 $\mathcal{C}$ ，由于一个二叉树要么为空，要么是左子树、根、右子树组合而成。

- 卡特兰数有个广为人知的组合意义， C_n 表示 n 个点二叉树的个数。
- 设卡特兰数的组合类为 $\mathcal{C}$ ，由于一个二叉树要么为空，要么是左子树、根、右子树组合而成。
-

$$\mathcal{C} = \mathcal{E} + \mathcal{C} \times \mathcal{Z} \times \mathcal{C}$$

- 卡特兰数有个广为人知的组合意义， C_n 表示 n 个点二叉树的个数。
- 设卡特兰数的组合类为 $\mathcal{C}$ ，由于一个二叉树要么为空，要么是左子树、根、右子树组合而成。

$$\mathcal{C} = \mathcal{E} + \mathcal{C} \times \mathcal{Z} \times \mathcal{C}$$

- 对应到生成函数， $C(z) = 1 + C^2(z) \times z$ 。

-

$$\mathcal{A} + \mathcal{B} = (\mathcal{A} \times \mathcal{E}_0) \cup (\mathcal{A} \times \mathcal{E}_1)$$

-

$$\mathcal{A} + \mathcal{B} = (\mathcal{A} \times \mathcal{E}_0) \cup (\mathcal{A} \times \mathcal{E}_1)$$

- 其中 $\mathcal{E}_0$ 与 $\mathcal{E}_1$ 为两个只包含一个大小为 0 对象的组合类，起到区分 $\mathcal{A}, \mathcal{B}$ 中元素的作用。

- 对于组合类 $\mathcal{A}$, 定义 “生成不定长序列” 构造为:

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 对于组合类 $\mathcal{A}$, 定义 “生成不定长序列” 构造为:

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 又作 $\text{SEQ}(\mathcal{A}) = \{(a_1, a_2, \dots, a_m) : m \geq 0, a \in \mathcal{A}\}$, 即用若干个 $\mathcal{A}$ 中元素拼成的有序序列。

- 对于组合类 $\mathcal{A}$, 定义“生成不定长序列”构造为:

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 又作 $\text{SEQ}(\mathcal{A}) = \{(a_1, a_2, \dots, a_m) : m \geq 0, a \in \mathcal{A}\}$, 即用若干个 $\mathcal{A}$ 中元素拼成的有序序列。
- 这里要求 $[z^0]A(z) = 0$, 否则会在计数序列中产生无穷大。

- 对于组合类 $\mathcal{A}$, 定义 “生成不定长序列” 构造为:

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 又作 $\text{SEQ}(\mathcal{A}) = \{(a_1, a_2, \dots, a_m) : m \geq 0, a \in \mathcal{A}\}$, 即用若干个 $\mathcal{A}$ 中元素拼成的有序序列。
- 这里要求 $[z^0]A(z) = 0$, 否则会在计数序列中产生无穷大。
- 写成生成函数, 则有:

$$B = \text{SEQ}(\mathcal{A}) \Rightarrow B(z) = \sum_{k \geq 0} A(z)^k = \frac{1}{1 - A(z)}$$

- 再来看 01 串，仍然设 S 为 01 串的组合类。

- 再来看 01 串，仍然设 S 为 01 串的组合类。
- 设 $\mathcal{C} = \{0, 1\}$ 。01 串可以看作 0, 1 生成的序列，即 $S = \mathcal{C}$ 。

- 再来看 01 串，仍然设 S 为 01 串的组合类。
- 设 $\mathcal{C} = \{0, 1\}$ 。01 串可以看作 0, 1 生成的序列，即 $S = \mathcal{C}$ 。
- 直接得到 $S(z) = \frac{1}{1 - 2z}$ 。

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 B 中若干元素的组合。

置换群下的等价类

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 $\mathcal{B}$ 中若干元素的组合。
- 定义 $a \in \mathcal{A}$ 在 $\mathcal{B}$ 上的拆分为 $a = (b_1, b_2, \dots, b_m)$, $b \in \mathcal{B}$ 。(这一般应是唯一的)

置换群下的等价类

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 B 中若干元素的组合。
- 定义 $a \in \mathcal{A}$ 在 B 上的拆分为 $a = (b_1, b_2, \dots, b_m), b \in B$ 。(这一般应是唯一的)
- 记 a 的容为 $\text{cap}_B(a) = m$ ，即拆分中的元素个数。

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 B 中若干元素的组合。
- 定义 $a \in \mathcal{A}$ 在 B 上的拆分为 $a = (b_1, b_2, \dots, b_m), b \in B$ 。(这一般应是唯一的)
- 记 a 的容为 $\text{cap}_B(a) = m$ ，即拆分中的元素个数。
- 令 $G = \{G_0, G_1, G_2, \dots\}$ 为一个置换群列，其中 G_k 包含若干大小为 k 的置换。

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 B 中若干元素的组合。
- 定义 $a \in \mathcal{A}$ 在 B 上的拆分为 $a = (b_1, b_2, \dots, b_m), b \in B$ 。(这一般应是唯一的)
- 记 a 的容为 $\text{cap}_B(a) = m$ ，即拆分中的元素个数。
- 令 $G = \{G_0, G_1, G_2, \dots\}$ 为一个置换群列，其中 G_k 包含若干大小为 k 的置换。
- 元素 a_1, a_2 在 B 上拆分后在 G 的作用下本质相同（等价），当且仅当：

置换群下的等价类

- 对于组合类 $\mathcal{A}$ ，其中的所有对象都是 $\mathcal{B}$ 中若干元素的组合。
- 定义 $a \in \mathcal{A}$ 在 $\mathcal{B}$ 上的拆分为 $a = (b_1, b_2, \dots, b_m)$, $b \in \mathcal{B}$ 。(这一般应是唯一的)
- 记 a 的容为 $\text{cap}_{\mathcal{B}}(a) = m$ ，即拆分中的元素个数。
- 令 $G = \{G_0, G_1, G_2, \dots\}$ 为一个置换群列，其中 G_k 包含若干大小为 k 的置换。
- 元素 a_1, a_2 在 $\mathcal{B}$ 上拆分后在 G 的作用下本质相同（等价），当且仅当：
- $\text{cap}_{\mathcal{B}}(a_1) = \text{cap}_{\mathcal{B}}(a_2)$ ，且 $\exists g \in G_{\text{cap}_{\mathcal{B}}(a_1)}$ ， $g(a_1) = a_2$ 。这里的置换 g 移动拆分中的元素。

- 定义 $\mathcal{A}/G_B$ 为一个（组合类的）组合类。

- 定义 $\mathcal{A}/G_B$ 为一个（组合类的）组合类。
- 对于 $a \in \mathcal{A}/G_B$, $a \subseteq \mathcal{A}$ 且 a 中的元素均本质相同。

- 定义 $\mathcal{A}/G_B$ 为一个（组合类的）组合类。
- 对于 $a \in \mathcal{A}/G_B$, $a \subseteq \mathcal{A}$ 且 a 中的元素均本质相同。
- 若 $a_1 \neq a_2$, 则 a_1, a_2 中的元素两两不等价。

置换群下的等价类

- 定义 $\mathcal{A}/G_B$ 为一个（组合类的）组合类。
- 对于 $a \in \mathcal{A}/G_B$, $a \subseteq \mathcal{A}$ 且 a 中的元素均本质相同。
- 若 $a_1 \neq a_2$, 则 a_1, a_2 中的元素两两不等价。
- 将 a 称为一个等价类, $|a|$ 即为 a 中任意一个元素的大小。

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：
-

$$\text{CYC}(\mathcal{A}) = (\text{SEQ}(\mathcal{A}) - \mathcal{E})/G_{\mathcal{A}}$$

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：
-

$$\text{CYC}(\mathcal{A}) = (\text{SEQ}(\mathcal{A}) - \mathcal{E})/G_{\mathcal{A}}$$

- 为了便于理解，可以将 $\mathcal{A}$ 中的元素看作珠子，而 $\text{CYC}(\mathcal{A})$ 中的元素就是生成的项链。两条项链若能通过旋转变得相同则本质相同。

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：
-

$$\text{CYC}(\mathcal{A}) = (\text{SEQ}(\mathcal{A}) - \mathcal{E})/G_{\mathcal{A}}$$

- 为了便于理解，可以将 $\mathcal{A}$ 中的元素看作珠子，而 $\text{CYC}(\mathcal{A})$ 中的元素就是生成的项链。两条项链若能通过旋转变得相同则本质相同。
- 先分析 k 个珠子组成的环，记该组合类为 $\text{CYC}_k(\mathcal{A}) = (\mathcal{A}^k)/G_{\mathcal{A}}$ ，生成函数为 $C_k(z)$ 。

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：
-

$$\text{CYC}(\mathcal{A}) = (\text{SEQ}(\mathcal{A}) - \mathcal{E})/G_{\mathcal{A}}$$

- 为了便于理解，可以将 $\mathcal{A}$ 中的元素看作珠子，而 $\text{CYC}(\mathcal{A})$ 中的元素就是生成的项链。两条项链若能通过旋转变得相同则本质相同。
- 先分析 k 个珠子组成的环，记该组合类为 $\text{CYC}_k(\mathcal{A}) = (\mathcal{A}^k)/G_{\mathcal{A}}$ ，生成函数为 $C_k(z)$ 。
- 根据 Burnside 引理，枚举 G_k 中旋转置换的步数 i 。

- 令 G 为全体环置换组成的置换群列。定义“生成（无标号）环构造”为：
-

$$\text{CYC}(\mathcal{A}) = (\text{SEQ}(\mathcal{A}) - \mathcal{E})/G_{\mathcal{A}}$$

- 为了便于理解，可以将 $\mathcal{A}$ 中的元素看作珠子，而 $\text{CYC}(\mathcal{A})$ 中的元素就是生成的项链。两条项链若能通过旋转变得相同则本质相同。
- 先分析 k 个珠子组成的环，记该组合类为 $\text{CYC}_k(\mathcal{A}) = (\mathcal{A}^k)/G_{\mathcal{A}}$ ，生成函数为 $C_k(z)$ 。
- 根据 Burnside 引理，枚举 G_k 中旋转置换的步数 i 。
-

$$\begin{aligned} C_k(z) &= \frac{1}{k} \sum_{i=0}^{k-1} A(z^{k/\gcd(i,k)})^{\gcd(i,k)} \\ &= \frac{1}{k} \sum_{d|k} \varphi(d) A(z^d)^{k/d} \end{aligned}$$

- 对各个 k 求和可得所有环的生成函数：

- 对各个 k 求和可得所有环的生成函数：

-

$$\begin{aligned}C(z) &= \sum_{k=1} \frac{1}{k} \sum_{d|k} \varphi(d) A(z^d)^{k/d} \\&= \sum_{d=1} \sum_{d|k} \frac{\varphi(d)}{k} A(z^d)^{k/d} \\&= \sum_{d=1} \frac{\varphi(d)}{d} \sum_{t=1} \frac{A(z^d)^t}{t} \\&= \sum_{d=1} \frac{\varphi(d)}{d} \ln \left(\frac{1}{1 - A(z^d)} \right)\end{aligned}$$

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：

Multiset 构造

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：
-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

Multiset 构造

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：

-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。

Multiset 构造

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：
-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：
-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。
- 所以, 等价类个数可以转化为字典序不降组合的数目。

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：
-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。
- 所以, 等价类个数可以转化为字典序不降组合的数目。
- 于是可以按照某种顺序枚举 $\mathcal{A}$ 中所有对象, 并一次性加入若干个:

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：

-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。
- 所以, 等价类个数可以转化为字典序不降组合的数目。
- 于是可以按照某种顺序枚举 $\mathcal{A}$ 中所有对象, 并一次性加入若干个:
-

$$\mathcal{B} = \text{MSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} \sum_{k=0} \{a\}^k$$

Multiset 构造

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：

-

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。
- 所以, 等价类个数可以转化为字典序不降组合的数目。
- 于是可以按照某种顺序枚举 $\mathcal{A}$ 中所有对象, 并一次性加入若干个:

-

$$\mathcal{B} = \text{MSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} \sum_{k=0} \{a\}^k$$

- 改写为生成函数, 则有:

Multiset 构造

- 令 G 为全体置换组成的置换群列。定义“生成（无序）集合构造”为：

$$\text{MSET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 在 G 下, $(b_1, b_2, b_3), (b_1, b_3, b_2), (b_3, b_1, b_2)$ 等都会被看作本质相同的组合。
- 不妨给每个元素一个字典序, 将组合按照字典序排序后去重即可。
- 所以, 等价类个数可以转化为字典序不降组合的数目。
- 于是可以按照某种顺序枚举 $\mathcal{A}$ 中所有对象, 并一次性加入若干个:

$$\mathcal{B} = \text{MSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} \sum_{k=0} \{a\}^k$$

- 改写为生成函数, 则有:

$$B(z) = \prod_{a \in \mathcal{A}} \sum_{k=0} z^{|a|k} = \prod_{a \in \mathcal{A}} \left(\frac{1}{1 - z^{|a|}} \right) = \prod_{i=1} \left(\frac{1}{1 - z^i} \right)^{[z^i]A(z)}$$

-

$$B(z) = \exp \left(\sum_{i=1} [z^i] A(z) \sum_{j=1} \frac{z^{ij}}{j} \right)$$

-

$$B(z) = \exp \left(\sum_{i=1} [z^i] A(z) \sum_{j=1} \frac{z^{ij}}{j} \right)$$

- 稍作代换可得 $B(z) = \exp \left(\sum_{i=1} \frac{A(z^i)}{i} \right)$ 。

-

$$B(z) = \exp \left(\sum_{i=1} [z^i] A(z) \sum_{j=1} \frac{z^{ij}}{j} \right)$$

- 稍作代换可得 $B(z) = \exp \left(\sum_{i=1} \frac{A(z^i)}{i} \right)$ 。
- 记 $\text{Exp}(A(z)) = B(z)$ ，称为 Polya 指数。记逆变换为 Ln 。

- 定义“生成（无序） k -集合构造”为：

- 定义 “生成（无序）k-集合构造” 为：
-

$$\text{MSET}_k(\mathcal{A}) = \mathcal{A}^k / G_{\mathcal{A}}$$

- 定义“生成（无序）k-集合构造”为：

-

$$\text{MSET}_k(\mathcal{A}) = \mathcal{A}^k / G_{\mathcal{A}}$$

- 考虑 Burnside 引理，记 $\text{cir}(g)$ 为置换 g 的环大小集合（多重集）。

- 定义“生成（无序）k-集合构造”为：

-

$$\text{MSET}_k(\mathcal{A}) = \mathcal{A}^k / G_{\mathcal{A}}$$

- 考虑 Burnside 引理，记 $\text{cir}(g)$ 为置换 g 的环大小集合（多重集）。

-

$$\mathcal{B} = \text{MSET}_k(\mathcal{A}) \Rightarrow B(z) = \frac{1}{k!} \sum_{g \in G_k} \prod_{i \in \text{cir}(g)} A(z^i)$$

- 定义“生成（无序）k-集合构造”为：

-

$$\text{MSET}_k(\mathcal{A}) = \mathcal{A}^k / G_{\mathcal{A}}$$

- 考虑 Burnside 引理，记 $\text{cir}(g)$ 为置换 g 的环大小集合（多重集）。

-

$$\mathcal{B} = \text{MSET}_k(\mathcal{A}) \Rightarrow B(z) = \frac{1}{k!} \sum_{g \in G_k} \prod_{i \in \text{cir}(g)} A(z^i)$$

- 朴素的计算，置换高达 $k!$ 个，效率低下。

- 定义“生成（无序）k-集合构造”为：

-

$$\text{MSET}_k(\mathcal{A}) = \mathcal{A}^k / G_{\mathcal{A}}$$

- 考虑 Burnside 引理，记 $\text{cir}(g)$ 为置换 g 的环大小集合（多重集）。

-

$$\mathcal{B} = \text{MSET}_k(\mathcal{A}) \Rightarrow B(z) = \frac{1}{k!} \sum_{g \in G_k} \prod_{i \in \text{cir}(g)} A(z^i)$$

- 朴素的计算，置换高达 $k!$ 个，效率低下。
- 注意到本质不同的 $\text{cir}(g)$ 是拆分数级别的，即可简化计算。

- 定义“生成（无序）幂集构造”为：

- 定义“生成（无序）幂集构造”为：
-

$$\text{PSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} (\{\epsilon\} + \{a\})$$

- 定义“生成（无序）幂集构造”为：

-

$$\text{PSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} (\{\epsilon\} + \{a\})$$

- 即枚举每个元素是否存在。可以粗略地理解为 $\text{PSET}(\mathcal{A}) = \{S \subseteq \mathcal{A}\}$ 。

- 定义“生成（无序）幂集构造”为：

-

$$\text{PSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} (\{\epsilon\} + \{a\})$$

- 即枚举每个元素是否存在。可以粗略地理解为 $\text{PSET}(\mathcal{A}) = \{S \subseteq \mathcal{A}\}$ 。
- 实际上，PSET 的组合意义就是 01 背包。（而 MSET 即完全背包）

- 定义“生成（无序）幂集构造”为：
-

$$\text{PSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} (\{\epsilon\} + \{a\})$$

- 即枚举每个元素是否存在。可以粗略地理解为 $\text{PSET}(\mathcal{A}) = \{S \subseteq \mathcal{A}\}$ 。
- 实际上，PSET 的组合意义就是 01 背包。（而 MSET 即完全背包）
- 将组合式子改写成生成函数，则有：

$$\begin{aligned} \mathcal{B} = \text{PSET}(\mathcal{A}) &\Rightarrow B(z) = \prod_{a \in \mathcal{A}} (1 + z^{|a|}) = \prod_{i=0} (1 + z^i)^{[z^i]A(z)} \\ &= \exp \left(\sum_{k=1} \frac{(-1)^{k-1} A(z^k)}{k} \right) \end{aligned}$$

- 定义“生成（无序）幂集构造”为：

-

$$\text{PSET}(\mathcal{A}) = \prod_{a \in \mathcal{A}} (\{\epsilon\} + \{a\})$$

- 即枚举每个元素是否存在。可以粗略地理解为 $\text{PSET}(\mathcal{A}) = \{S \subseteq \mathcal{A}\}$ 。
- 实际上，PSET 的组合意义就是 01 背包。（而 MSET 即完全背包）
- 将组合式子改写成生成函数，则有：

$$\mathcal{B} = \text{PSET}(\mathcal{A}) \Rightarrow B(z) = \prod_{a \in \mathcal{A}} (1 + z^{|a|}) = \prod_{i=0} (1 + z^i)^{[z^i]A(z)}$$

$$= \exp \left(\sum_{k=1} \frac{(-1)^{k-1} A(z^k)}{k} \right)$$

- 记 $\overline{\text{Exp}}(A(z)) = B(z)$ ，称为改版 Polya 指数，记逆变换为 $\overline{\text{Ln}}$ 。

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。
- 若填写的标号恰好为 $1 \sim |a|$ 的排列，则称为（强）标号的。若不一定，则称弱标号的。记 $\text{ind}(a)$ 为 a 的标号集合。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。
- 若填写的标号恰好为 $1 \sim |a|$ 的排列，则称为（强）标号的。若不一定，则称弱标号的。记 $\text{ind}(a)$ 为 a 的标号集合。
- 我们定义组合对象 a, b 的组合积 $a \otimes b$ ，即 a 与 b 组合能构造出什么组合对象。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。
- 若填写的标号恰好为 $1 \sim |a|$ 的排列，则称为（强）标号的。若不一定，则称弱标号的。记 $\text{ind}(a)$ 为 a 的标号集合。
- 我们定义组合对象 a, b 的组合积 $a \otimes b$ ，即 a 与 b 组合能构造出什么组合对象。
- 在无标号体系中，我们将 $a \otimes b$ 简单地定义为笛卡尔积 (a, b) ；在有标号体系中，情况将更加复杂。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。
- 若填写的标号恰好为 $1 \sim |a|$ 的排列，则称为（强）标号的。若不一定，则称弱标号的。记 $\text{ind}(a)$ 为 a 的标号集合。
- 我们定义组合对象 a, b 的组合积 $a \otimes b$ ，即 a 与 b 组合能构造出什么组合对象。
- 在无标号体系中，我们将 $a \otimes b$ 简单地定义为笛卡尔积 (a, b) ；在有标号体系中，情况将更加复杂。
- 设 $\rho(a)$ 为将组合对象 a 的标号离散化后的结果，例如 $\rho(\{1, 5, 7\}) = \{1, 2, 3\}$ 。

有标号体系

- 对于一个带标号的组合对象 a ，其带的标号数量恰好为 $|a|$ 。
- 你可以理解为 a 恰好拥有 $|a|$ 个“节点”、“格子”等容器（具体是什么我们并不在意），每个容器恰好可以填写一个标号。
- 若填写的标号恰好为 $1 \sim |a|$ 的排列，则称为（强）标号的。若不一定，则称弱标号的。记 $\text{ind}(a)$ 为 a 的标号集合。
- 我们定义组合对象 a, b 的组合积 $a \otimes b$ ，即 a 与 b 组合能构造出什么组合对象。
- 在无标号体系中，我们将 $a \otimes b$ 简单地定义为笛卡尔积 (a, b) ；在有标号体系中，情况将更加复杂。
- 设 $\rho(a)$ 为将组合对象 a 的标号离散化后的结果，例如 $\rho(\{1, 5, 7\}) = \{1, 2, 3\}$ 。
- 形式化地，我们将 $a \otimes b$ 定义为：

$$a \otimes b = \{(a', b') : \rho(a') = a, \rho(b') = b, (a', b') \text{ 是强标号的}\}$$

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。
- 我们来具体看一下其中 $\square\blacksquare\blacksquare\square$ 是怎么产生的。

有标号体系

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。
- 我们来具体看一下其中 $\square\blacksquare\blacksquare\square$ 是怎么产生的。
- $a = \square\square, b = \blacksquare\blacksquare$, a 的标号为 $\{1, 2\}$, b 的标号为 $\{1, 2\}$ 。

有标号体系

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。
- 我们来具体看一下其中 $\square\blacksquare\blacksquare\square$ 是怎么产生的。
- $a = \square\square, b = \blacksquare\blacksquare$, a 的标号为 $\{1, 2\}$, b 的标号为 $\{1, 2\}$ 。
- 设 $\circ$ 表示间隔符, $a' = \square \circ \circ \square$, a' 的标号为 $\{1, 4\}$, $\rho(a') = \{1, 2\}$;
 $b' = \circ \blacksquare \blacksquare \circ$, b' 的标号为 $\{2, 3\}$, $\rho(b') = \{1, 2\}$, 而 $(\{1, 4\}, \{2, 3\})$ 也是强标号的。

有标号体系

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。
- 我们来具体看一下其中 $\square\blacksquare\blacksquare\square$ 是怎么产生的。
- $a = \square\square, b = \blacksquare\blacksquare$, a 的标号为 $\{1, 2\}$, b 的标号为 $\{1, 2\}$ 。
- 设 $\circ$ 表示间隔符, $a' = \square \circ \square$, a' 的标号为 $\{1, 4\}$, $\rho(a') = \{1, 2\}$;
 $b' = \circ\blacksquare\blacksquare\circ$, b' 的标号为 $\{2, 3\}$, $\rho(b') = \{1, 2\}$, 而 $(\{1, 4\}, \{2, 3\})$ 也是强标号的。
- 带标号的组合对象使用 EGF 来刻画其生成函数, 设 A_i 表示 $\mathcal{A}$ 中有多少个组合对象的大小为 i 。

有标号体系

- 例如：

$$\square\square \otimes \blacksquare\blacksquare = \{\square\square\blacksquare\blacksquare, \square\blacksquare\square\blacksquare, \square\blacksquare\blacksquare\square, \blacksquare\square\square\blacksquare, \blacksquare\square\blacksquare\square, \blacksquare\blacksquare\square\square\}$$

- 这也正是将两个序列归并起来的所有方案。
- 我们来具体看一下其中 $\square\blacksquare\blacksquare\square$ 是怎么产生的。
- $a = \square\square, b = \blacksquare\blacksquare$, a 的标号为 $\{1, 2\}$, b 的标号为 $\{1, 2\}$ 。
- 设 $\circ$ 表示间隔符, $a' = \square \circ \circ \square$, a' 的标号为 $\{1, 4\}$, $\rho(a') = \{1, 2\}$;
 $b' = \circ \blacksquare \blacksquare \circ$, b' 的标号为 $\{2, 3\}$, $\rho(b') = \{1, 2\}$, 而 $(\{1, 4\}, \{2, 3\})$ 也是强标号的。
- 带标号的组合对象使用 EGF 来刻画其生成函数, 设 A_i 表示 $\mathcal{A}$ 中有多少个组合对象的大小为 i 。
-

$$A(z) = \sum_{a \in \mathcal{A}} \frac{z^{|a|}}{|a|!} = \sum_{i=0} \frac{A_i z^i}{i!}$$

- 其实和无标号是一样的，定义“生成序列构造”为：

- 其实和无标号是一样的，定义“生成序列构造”为：
-

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 其实和无标号是一样的，定义“生成序列构造”为：

-

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 要求 $A_0 = 0$ ，否则会在计数中产生无穷大。

- 其实和无标号是一样的，定义“生成序列构造”为：

-

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 要求 $A_0 = 0$ ，否则会在计数中产生无穷大。

-

$$\text{SEQ}(\mathcal{A}) = \{a_1 \otimes a_2 \otimes \dots : a \in \mathcal{A}\}$$

- 其实和无标号是一样的，定义“生成序列构造”为：

-

$$\text{SEQ}(\mathcal{A}) = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \cdots = \sum_{k \geq 0} \mathcal{A}^k$$

- 要求 $A_0 = 0$ ，否则会在计数中产生无穷大。

-

$$\text{SEQ}(\mathcal{A}) = \{a_1 \otimes a_2 \otimes \cdots : a \in \mathcal{A}\}$$

-

$$\mathcal{B} = \text{SEQ}(\mathcal{A}) \Rightarrow B(z) = \frac{1}{1 - A(z)}$$

- 设 $\mathcal{C}_k$ 表示 $\mathcal{A}$ 中大小为 k 的组合对象构成的组合类。

- 设 $\mathcal{C}_k$ 表示 $\mathcal{A}$ 中大小为 k 的组合对象构成的组合类。
-

$$\text{PNT}(\mathcal{A}) = \sum_{k=0} \mathcal{C}_k \times \{\epsilon_1, \epsilon_2, \dots, \epsilon_k\}$$

- 设 $\mathcal{C}_k$ 表示 $\mathcal{A}$ 中大小为 k 的组合对象构成的组合类。
-

$$\text{PNT}(\mathcal{A}) = \sum_{k=0} \mathcal{C}_k \times \{\epsilon_1, \epsilon_2, \dots, \epsilon_k\}$$

- 组合意义是在图 a 中选出一个特殊点（根）。

- 设 $\mathcal{C}_k$ 表示 $\mathcal{A}$ 中大小为 k 的组合对象构成的组合类。
-

$$\text{PNT}(\mathcal{A}) = \sum_{k=0} \mathcal{C}_k \times \{\epsilon_1, \epsilon_2, \dots, \epsilon_k\}$$

- 组合意义是在图 a 中选出一个特殊点（根）。
- 若 $\mathcal{B} = \text{PNT}(\mathcal{A})$ ，则 $[z^n]\mathcal{B}(z) = n[z^n]\mathcal{A}(z)$ 。

- 设 $\mathcal{C}_k$ 表示 $\mathcal{A}$ 中大小为 k 的组合对象构成的组合类。
-

$$\text{PNT}(\mathcal{A}) = \sum_{k=0} \mathcal{C}_k \times \{\epsilon_1, \epsilon_2, \dots, \epsilon_k\}$$

- 组合意义是在图 a 中选出一个特殊点（根）。
- 若 $\mathcal{B} = \text{PNT}(\mathcal{A})$, 则 $[z^n]\mathcal{B}(z) = n[z^n]\mathcal{A}(z)$ 。
-

$$\mathcal{B} = \text{PNT}(\mathcal{A}) \Rightarrow B(z) = zA'(z)$$

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：
-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

Set 构造

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 先来考虑容为 k 的生成集合，设为 $\text{SET}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ 。

Set 构造

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 先来考虑容为 k 的生成集合，设为 $\text{SET}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ 。
- 在有标号序列中，任意两个元素都是不同的（因为标号集合不交）。所以一种方案通过任意非单位置换后都和原来不同，故等价类的大小达到上界 $k!$ （即 $|G_k|$ ）。

Set 构造

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 先来考虑容为 k 的生成集合，设为 $\text{SET}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ 。
- 在有标号序列中，任意两个元素都是不同的（因为标号集合不交）。所以一种方案通过任意非单位置换后都和原来不同，故等价类的大小达到上界 $k!$ （即 $|G_k|$ ）。
- 可得：
$$A_k(z) = \frac{A(z)^k}{k!}。$$

Set 构造

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 先来考虑容为 k 的生成集合，设为 $\text{SET}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ 。
- 在有标号序列中，任意两个元素都是不同的（因为标号集合不交）。所以一种方案通过任意非单位置换后都和原来不同，故等价类的大小达到上界 $k!$ （即 $|G_k|$ ）。
- 可得： $A_k(z) = \frac{A(z)^k}{k!}$ 。
- 对所有 k 求和可得：

$$\mathcal{B} = \text{SET}(\mathcal{A}) \Rightarrow B(z) = \sum_{k=0} \frac{A(z)^k}{k!} = \exp A(z)$$

Set 构造

- 记 G 为全体置换组成的置换群组，将“生成集合构造”定义为：

-

$$\text{SET}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 先来考虑容为 k 的生成集合，设为 $\text{SET}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ 。
- 在有标号序列中，任意两个元素都是不同的（因为标号集合不交）。所以一种方案通过任意非单位置换后都和原来不同，故等价类的大小达到上界 $k!$ （即 $|G_k|$ ）。
- 可得： $A_k(z) = \frac{A(z)^k}{k!}$ 。
- 对所有 k 求和可得：

$$\mathcal{B} = \text{SET}(\mathcal{A}) \Rightarrow B(z) = \sum_{k=0} \frac{A(z)^k}{k!} = \exp A(z)$$

- 这也正是 $\exp$ 的组合意义。

有标号 Cycle 构造

- 记 G 为全体环置换组成的置换群组，将“生成环构造”定义为：

有标号 Cycle 构造

- 记 G 为全体环置换组成的置换群组，将“生成环构造”定义为：
-

$$\text{CYC}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

有标号 Cycle 构造

- 记 G 为全体环置换组成的置换群组，将“生成环构造”定义为：
-

$$\text{CYC}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 仍然先考虑容为 k 的环，设 $\text{CYC}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ ，由于带标号，任意两个元素都是不同的，所以直接算完后除以 k 即可。

有标号 Cycle 构造

- 记 G 为全体环置换组成的置换群组，将“生成环构造”定义为：

-

$$\text{CYC}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 仍然先考虑容为 k 的环，设 $\text{CYC}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ ，由于带标号，任意两个元素都是不同的，所以直接算完后除以 k 即可。

-

$$A_k(z) = \frac{A(z)^k}{k}$$

有标号 Cycle 构造

- 记 G 为全体环置换组成的置换群组，将“生成环构造”定义为：

-

$$\text{CYC}(\mathcal{A}) = \text{SEQ}(\mathcal{A})/G_{\mathcal{A}}$$

- 仍然先考虑容为 k 的环，设 $\text{CYC}_k(\mathcal{A}) = \mathcal{A}^k/G_{\mathcal{A}}$ ，生成函数设为 $A_k(z)$ ，由于带标号，任意两个元素都是不同的，所以直接算完后除以 k 即可。

-

$$A_k(z) = \frac{A(z)^k}{k}$$

- 对所有 k 求和可得：

$$B = \text{CYC}(\mathcal{A}) \Rightarrow B(z) = \sum_{k=1} \frac{A(z)^k}{k} = \ln \left(\frac{1}{1 - A(z)} \right)$$

- 定义“删除构造”以及“加入构造”为：

- 定义“删除构造”以及“加入构造”为：
-

$$\text{DEL}(\mathcal{A}) = \{b : |b| = |a| - 1, a \in \mathcal{A}\}$$

$$\text{ADD}(\mathcal{A}) = \{b : |b| = |a| + 1, a \in \mathcal{A}\}$$

- 定义“删除构造”以及“加入构造”为：

-

$$\text{DEL}(\mathcal{A}) = \{b : |b| = |a| - 1, a \in \mathcal{A}\}$$

$$\text{ADD}(\mathcal{A}) = \{b : |b| = |a| + 1, a \in \mathcal{A}\}$$

- 组合意义就是将所有组合对象“删除” / “加入”一个点，使得这个点“参与” / “不参与”标号的分配。

- 定义“删除构造”以及“加入构造”为：

-

$$\text{DEL}(\mathcal{A}) = \{b : |b| = |a| - 1, a \in \mathcal{A}\}$$

$$\text{ADD}(\mathcal{A}) = \{b : |b| = |a| + 1, a \in \mathcal{A}\}$$

- 组合意义就是将所有组合对象“删除” / “加入”一个点，使得这个点“参与” / “不参与”标号的分配。
- 若 $A_0 = 0$ ，则有 $\text{ADD}(\text{DEL}(\mathcal{A})) = \mathcal{A}$ 。

- 定义“删除构造”以及“加入构造”为：

-

$$\text{DEL}(\mathcal{A}) = \{b : |b| = |a| - 1, a \in \mathcal{A}\}$$

$$\text{ADD}(\mathcal{A}) = \{b : |b| = |a| + 1, a \in \mathcal{A}\}$$

- 组合意义就是将所有组合对象“删除” / “加入”一个点，使得这个点“参与” / “不参与”标号的分配。
- 若 $A_0 = 0$ ，则有 $\text{ADD}(\text{DEL}(\mathcal{A})) = \mathcal{A}$ 。
- 比如若 $\mathcal{C} = \text{ADD}(\text{DEL}(\mathcal{A}) \times \mathcal{B})$ ，则可以理解为 $\mathcal{A}$ 将标号 1 藏起来，不参与分配，最后再将其加回。即标号 1 一定分配给了 $\mathcal{A}$ 中对象。

- 定义“删除构造”以及“加入构造”为：

-

$$\text{DEL}(\mathcal{A}) = \{b : |b| = |a| - 1, a \in \mathcal{A}\}$$

$$\text{ADD}(\mathcal{A}) = \{b : |b| = |a| + 1, a \in \mathcal{A}\}$$

- 组合意义就是将所有组合对象“删除” / “加入”一个点，使得这个点“参与” / “不参与”标号的分配。
- 若 $A_0 = 0$ ，则有 $\text{ADD}(\text{DEL}(\mathcal{A})) = \mathcal{A}$ 。
- 比如若 $\mathcal{C} = \text{ADD}(\text{DEL}(\mathcal{A}) \times \mathcal{B})$ ，则可以理解为 $\mathcal{A}$ 将标号 1 藏起来，不参与分配，最后再将其加回。即标号 1 一定分配给了 $\mathcal{A}$ 中对象。
- 写成生成函数的形式，则有：

$$\mathcal{B} = \text{DEL}(\mathcal{A}) \Rightarrow B(z) = A'(z)$$

$$\mathcal{B} = \text{ADD}(\mathcal{A}) \Rightarrow B(z) = \int A(z) dz$$

组合计数例题

- 给出两个数 n, p ，你需要求出下面四个问题的答案：
 1. n 个点的带标号无根树数量。
 2. n 个点的带标号有根树数量。
 3. n 个点的无标号有根树数量。
 4. n 个点的无标号无根树数量。
- 答案对 p 取模。
- 多组询问，每次询问给出 k, n, p ， n 是点数， p 是模数， k 表示第几种询问。
- 对于 1, 2 询问，保证 $1 \leq n \leq 10^9$ 。
- 对于 3, 4 询问，保证 $1 \leq n \leq 10^3$ 且 $n \leq p$ ($\Theta(n^2)$ 能过)。
- $2 \leq p \leq 10^4$ 且 p 为质数。

- 对于 1 询问，答案为 n^{n-2} ；对于 2 询问，答案为 n^{n-1} 。

- 对于 1 询问, 答案为 n^{n-2} ; 对于 2 询问, 答案为 n^{n-1} 。
- 对于 3 询问, 设非空无标号有根树的组合类为 $\mathcal{T}$ 。

- 对于 1 询问, 答案为 n^{n-2} ; 对于 2 询问, 答案为 n^{n-1} 。
- 对于 3 询问, 设非空无标号有根树的组合类为 $\mathcal{T}$ 。
-

$$\mathcal{T} = \mathcal{Z} + \mathcal{Z} \times \text{MSET}(\mathcal{T})$$

- 对于 1 询问，答案为 n^{n-2} ；对于 2 询问，答案为 n^{n-1} 。
- 对于 3 询问，设非空无标号有根树的组合类为 $\mathcal{T}$ 。
-

$$\mathcal{T} = \mathcal{Z} + \mathcal{Z} \times \text{MSET}(\mathcal{T})$$

- 写成生成函数的形式：

- 对于 1 询问, 答案为 n^{n-2} ; 对于 2 询问, 答案为 n^{n-1} 。
- 对于 3 询问, 设非空无标号有根树的组合类为 $\mathcal{T}$ 。
-

$$\mathcal{T} = \mathcal{Z} + \mathcal{Z} \times \text{MSET}(\mathcal{T})$$

- 写成生成函数的形式:
-

$$T(z) = z + z \exp \left(\sum_{i=1} \sum_{j=1} \frac{[z^i]T(z)z^{ij}}{j} \right)$$

- 对于 1 询问, 答案为 n^{n-2} ; 对于 2 询问, 答案为 n^{n-1} 。
- 对于 3 询问, 设非空无标号有根树的组合类为 $\mathcal{T}$ 。
-

$$\mathcal{T} = \mathcal{Z} + \mathcal{Z} \times \text{MSET}(\mathcal{T})$$

- 写成生成函数的形式:
-

$$T(z) = z + z \exp \left(\sum_{i=1} \sum_{j=1} \frac{[z^i]T(z)z^{ij}}{j} \right)$$

- 考虑递推, 设 $t_n = [z^n]T(z)$, 容易发现 t_n 只与 $t_1, \dots, t_{n-1}$ 有关。

- 直接递推复杂度是 $\Theta(n^3)$ 的，无法通过。

- 直接递推复杂度是 $\Theta(n^3)$ 的，无法通过。
- 发现维护多项式 $\sum_{i=1} \sum_{j=1} \frac{t_i z^{ij}}{j}$ 的复杂度是 $\Theta(n \log n)$ 的，瓶颈在于 $\Theta(n)$ 次多项式 exp。

- 直接递推复杂度是 $\Theta(n^3)$ 的，无法通过。
- 发现维护多项式 $\sum_{i=1} \sum_{j=1} \frac{t_i z^{ij}}{j}$ 的复杂度是 $\Theta(n \log n)$ 的，瓶颈在于 $\Theta(n)$ 次多项式 exp。
- 设 $F(z) = \exp \left(\sum_{i=1} \sum_{j=1} \frac{t_i z^{ij}}{j} \right)$ ，由于 $T(z) = z + zF(z)$ ，假设我们已经求出了 $t_1, \dots, t_{n-1}$ ，要求 t_n ，那么其实 $F(z)$ 的前 $n-2$ 项系数我们已经知道了，分别为 $t_2, \dots, t_{n-1}$ ，那么只需要算 $[z^{n-1}]F(z)$ 即可，这样时间复杂度降为 $\Theta(n^2)$ 。

- 直接递推复杂度是 $\Theta(n^3)$ 的，无法通过。
- 发现维护多项式 $\sum_{i=1} \sum_{j=1} \frac{t_i z^{ij}}{j}$ 的复杂度是 $\Theta(n \log n)$ 的，瓶颈在于 $\Theta(n)$ 次多项式 exp。
- 设 $F(z) = \exp \left(\sum_{i=1} \sum_{j=1} \frac{t_i z^{ij}}{j} \right)$ ，由于 $T(z) = z + zF(z)$ ，假设我们已经求出了 $t_1, \dots, t_{n-1}$ ，要求 t_n ，那么其实 $F(z)$ 的前 $n-2$ 项系数我们已经知道了，分别为 $t_2, \dots, t_{n-1}$ ，那么只需要算 $[z^{n-1}]F(z)$ 即可，这样时间复杂度降为 $\Theta(n^2)$ 。
- 对于 4 询问，先和 3 询问一样，算出 $t_1, \dots, t_n$ 。然后钦定一棵树的重心作为根。容斥，答案变成无标号有根树的数量减去无标号且根不为重心的树的数量。枚举根的子树大小 $> \lfloor n/2 \rfloor$ 的那棵，计算贡献。最后特判有两个重心的情况即可。

- 按如下方式可以生成一棵 n 个点的随机树：
- 以 1 号节点为根。
- 对于 $i \in [2, n]$ ，独立地从 $[1, i)$ 中等概率随机选取一个节点作为 i 的父亲。
- 通过上面的方式生成 k 棵 n 个节点的有根树 T_1 至 T_k ，求它们两两同构的概率，答案对质数 p 取模。
- 两棵 n 个节点的有根树 T_1 和 T_2 同构当且仅当存在长度为 n 的排列 p ，满足 $p_1 = 1$ ，且对于 $i \in [2, n]$ ，若 i 在 T_1 的父亲是 f ，则 p_i 在 T_2 的父亲是 p_f 。
- $1 \leq n \leq 2 \times 10^3, 1 \leq k \leq 10^9, 10^8 \leq p \leq 10^9$ 且保证 p 是质数。

- 这里给出个人认为最阳间的做法，只用到了一点 MSET 的思想。

- 这里给出个人认为最阳间的做法，只用到了一点 MSET 的思想。
- 设 $\mathcal{T}$ 无标号有根树的组合类， $\mathcal{T}_n$ 表示 n 个节点的无标号有根树的组合类， τ 表示组合对象。

- 这里给出个人认为最阳间的做法，只用到了一点 MSET 的思想。
- 设 $\mathcal{T}$ 无标号有根树的组合类， $\mathcal{T}_n$ 表示 n 个节点的无标号有根树的组合类， τ 表示组合对象。
- 设 $g(\tau)$ 表示给 τ 编号使其符合条件的方案数除以 $|\tau|!$ 后的结果。设 τ 中各节点子树大小分别为 S_i ，各节点子树构成的等价类大小分别为 $C_{i,1}, C_{i,2}, \dots$ ，容易得到 $g(\tau) = \prod_i \frac{1}{S_i} \prod_j \frac{1}{C_{i,j}!}$ 。

- 这里给出个人认为最阳间的做法，只用到了一点 MSET 的思想。
- 设 $\mathcal{T}$ 无标号有根树的组合类， $\mathcal{T}_n$ 表示 n 个节点的无标号有根树的组合类， τ 表示组合对象。
- 设 $g(\tau)$ 表示给 τ 编号使其符合条件的方案数除以 $|\tau|!$ 后的结果。设 τ 中各节点子树大小分别为 S_i ，各节点子树构成的等价类大小分别为 $C_{i,1}, C_{i,2}, \dots$ ，容易得到 $g(\tau) = \prod_i \frac{1}{S_i} \prod_j \frac{1}{C_{i,j}!}$ 。
- 答案就是 $\frac{1}{(n-1)!^k} \sum_{\tau \in \mathcal{T}_n} (n!g(\tau))^k = n^k \sum_{\tau \in \mathcal{T}_n} g(\tau)^k$ 。

- 这里给出个人认为最阳间的做法，只用到了一点 MSET 的思想。
- 设 $\mathcal{T}$ 无标号有根树的组合类， $\mathcal{T}_n$ 表示 n 个节点的无标号有根树的组合类， τ 表示组合对象。
- 设 $g(\tau)$ 表示给 τ 编号使其符合条件的方案数除以 $|\tau|!$ 后的结果。设 τ 中各节点子树大小分别为 S_i ，各节点子树构成的等价类大小分别为 $C_{i,1}, C_{i,2}, \dots$ ，容易得到 $g(\tau) = \prod_i \frac{1}{S_i} \prod_j \frac{1}{C_{i,j}!}$ 。
- 答案就是 $\frac{1}{(n-1)!^k} \sum_{\tau \in \mathcal{T}_n} (n!g(\tau))^k = n^k \sum_{\tau \in \mathcal{T}_n} g(\tau)^k$ 。
- 设 $f_{m,i}$ 表示 $\sum_{\tau \in \mathcal{T}_m} g(\tau)^{ik}$ ，则答案就是 $f_{n,1}n^k$ ，这里 i 的出现可能显得不自然，但是推到 MSET 的时候就会变得自然。

- 删去根后，变成了子树的 MSET。

- 删去根后，变成了子树的 MSET。
-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \prod_{\tau \in \mathcal{T}_j} \prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp}$$

- 删去根后，变成了子树的 MSET。

-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \prod_{\tau \in \mathcal{T}_j} \prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp}$$

- 发现有 $\prod_{\tau \in \mathcal{T}_j}$ 肯定不行，ln 后取 exp 将其转化成 $\sum$ ，然后考虑用 f 去代换。

- 删去根后，变成了子树的 MSET。

-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \prod_{\tau \in \mathcal{T}_j} \prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp}$$

- 发现有 $\prod_{\tau \in \mathcal{T}_j}$ 肯定不行，ln 后取 exp 将其转化成 $\sum$ ，然后考虑用 f 去代换。

-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{\tau \in \mathcal{T}_j} \ln \left(\prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp} \right) \right)$$

- 删去根后，变成了子树的 MSET。

-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \prod_{\tau \in \mathcal{T}_j} \prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp}$$

- 发现有 $\prod_{\tau \in \mathcal{T}_j}$ 肯定不行，ln 后取 exp 将其转化成 $\sum$ ，然后考虑用 f 去代换。

-

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{\tau \in \mathcal{T}_j} \ln \left(\prod_{p=0}^{\infty} \frac{g(\tau)^{ipk}}{(p!)^{ik}} x^{jp} \right) \right)$$

- 设 $H_i(x) = \ln \prod_{p=0}^{\infty} \frac{x^p}{(p!)^{ik}} = \sum_{p=0}^{\infty} h_{i,p} x^p$ 。

- 将 H 带入得：
$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{\tau \in \mathcal{T}_j} H_i(g(\tau)^{ik} x^j) \right) =$$
$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} \sum_{\tau \in \mathcal{T}_j} g(\tau)^{ipk} \right) =$$
$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} f_{j,ip} \right)。$$

- 将 H 带入得: $f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{\tau \in \mathcal{T}_j} H_i(g(\tau)^{ik} x^j) \right) =$

$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} \sum_{\tau \in \mathcal{T}_j} g(\tau)^{ipk} \right) =$$

$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} f_{j,ip} \right)。$$

- 由于必然有 $jp \leq m-1$, 所以 $jpi < mi$, 也就是说 $f_{a,b}$ 能转移到 $f_{c,d}$, 必须满足 $cd > ab$, 而最终答案只需要 $f_{n,1}$, 于是我们得出一个重要结论: $f_{i,j}$ 中只需取 $ij \leq n$ 的状态即可。

- 将 H 带入得：

$$f_{m,i} = \frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{\tau \in \mathcal{T}_j} H_i(g(\tau)^{ik} x^j) \right) =$$

$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} \sum_{\tau \in \mathcal{T}_j} g(\tau)^{ipk} \right) =$$

$$\frac{1}{m^{ik}} [x^{m-1}] \prod_{j=1}^{m-1} \exp \left(\sum_{p=0}^{\infty} h_{i,p} x^{jp} f_{j,ip} \right)。$$
- 由于必然有 $jp \leq m-1$ ，所以 $jpi < mi$ ，也就是说 $f_{a,b}$ 能转移到 $f_{c,d}$ ，必须满足 $cd > ab$ ，而最终答案只需要 $f_{n,1}$ ，于是我们得出一个重要结论： $f_{i,j}$ 中只需取 $ij \leq n$ 的状态即可。
- 首先考虑预处理 H 的步骤，由于 $H_i(x)$ 只需要处理 $\Theta\left(\frac{n}{i}\right)$ 个系数，所以复杂度为 $\Theta\left(\sum_{i=1}^n \frac{n^2}{i^2}\right) = \Theta(n^2)。$

- 然后考虑处理 $\exp$ 乘积的步骤，维护 n 个多项式 L_{j_0}

- 然后考虑处理 $\exp$ 乘积的步骤，维护 n 个多项式 L_i 。
- 每次算出 $f_{m,1}, f_{m,2}, \dots, f_{m, \lfloor \frac{n}{m} \rfloor}$ 之后，枚举 $i \in [1, \lfloor \frac{n}{m} \rfloor]$ 就能得到 $\exp\left(\sum_{p=0}^{\infty} h_{i,p} x^{mp} f_{m,ip}\right)$ ，然后令 L_i 乘上这个多项式，求 $\exp$ 部分的复杂度为 $\Theta\left(\sum_{i=1}^n \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n^2}{i^2 j^2}\right) = \Theta(n^2)$ ，更新 L_i 的复杂度为 $\Theta\left(\sum_{i=1}^n \frac{n}{i} \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n}{ij}\right) = \Theta(n^2 \log n)$ 。

- 然后考虑处理 $\exp$ 乘积的步骤，维护 n 个多项式 L_i 。
- 每次算出 $f_{m,1}, f_{m,2}, \dots, f_{m, \lfloor \frac{n}{m} \rfloor}$ 之后，枚举 $i \in [1, \lfloor \frac{n}{m} \rfloor]$ 就能得到 $\exp\left(\sum_{p=0}^{\infty} h_{i,p} x^{mp} f_{m,ip}\right)$ ，然后令 L_i 乘上这个多项式，求 $\exp$ 部分的复杂度为 $\Theta\left(\sum_{i=1}^n \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n^2}{i^2 j^2}\right) = \Theta(n^2)$ ，更新 L_i 的复杂度为 $\Theta\left(\sum_{i=1}^n \frac{n}{i} \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n}{ij}\right) = \Theta(n^2 \log n)$ 。
- 然后每次 $1, \dots, m-1$ 的贡献都算上之后，就能直接得出 $f_{m,1}, f_{m,2}, \dots$ 。

- 然后考虑处理 $\exp$ 乘积的步骤，维护 n 个多项式 L_i 。
- 每次算出 $f_{m,1}, f_{m,2}, \dots, f_{m, \lfloor \frac{n}{m} \rfloor}$ 之后，枚举 $i \in [1, \lfloor \frac{n}{m} \rfloor]$ 就能得到 $\exp\left(\sum_{p=0}^{\infty} h_{i,p} x^{mp} f_{m,ip}\right)$ ，然后令 L_i 乘上这个多项式，求 $\exp$ 部分的复杂度为 $\Theta\left(\sum_{i=1}^n \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n^2}{i^2 j^2}\right) = \Theta(n^2)$ ，更新 L_i 的复杂度为 $\Theta\left(\sum_{i=1}^n \frac{n}{i} \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} \frac{n}{ij}\right) = \Theta(n^2 \log n)$ 。
- 然后每次 $1, \dots, m-1$ 的贡献都算上之后，就能直接得出 $f_{m,1}, f_{m,2}, \dots$ 。
- 总时间复杂度 $\Theta(n^2 \log n)$ 。

SOS DP

- SOS DP，本质上就是高维前缀和。

- SOS DP，本质上就是高维前缀和。
- 一般就是对于一个大小为 2^n 的数组 a ，你要求出：

$$f_T = \sum_{S \subseteq T} a_S$$

- SOS DP，本质上就是高维前缀和。
- 一般就是对于一个大小为 2^n 的数组 a ，你要求出：

$$f_T = \sum_{S \subseteq T} a_S$$

- 直接在每个维度上做一遍前缀和即可，时间复杂度 $\Theta(2^n n)$ 。

- 求 $f_S = H_1 \left(\sum_{S \subseteq T} g_S \right), g_S = H_2 \left(\sum_{S \subseteq T} f_S \right)$ 。

- 求 $f_S = H_1 \left(\sum_{S \subset T} g_S \right), g_S = H_2 \left(\sum_{S \subseteq T} f_S \right)$ 。
- 其中 H_1, H_2 是某种简单函数。

- 求 $f_S = H_1 \left(\sum_{S \subset T} g_S \right), g_S = H_2 \left(\sum_{S \subseteq T} f_S \right)$ 。
- 其中 H_1, H_2 是某种简单函数。
- 直接设 $f_{S,i}, g_{S,i}$ 分别表示只做 S 前 i 维的前缀和的结果。

- 求 $f_S = H_1 \left(\sum_{S \subseteq T} g_S \right), g_S = H_2 \left(\sum_{S \subseteq T} f_S \right)$ 。
- 其中 H_1, H_2 是某种简单函数。
- 直接设 $f_{S,i}, g_{S,i}$ 分别表示只做 S 前 i 维的前缀和的结果。
- 先枚举 S , 再枚举 i 转移即可, 时空复杂度 $\Theta(2^n n)$ 。

- 定义一个字符串的美丽值为其所有极长连续段长度的平方和。
- 例如：aabbcaaa 的美丽值为 $2^2 + 2^2 + 1^2 + 3^2 = 18$ 。
- 给定一个长度为 n 的字符串 s ，字符集为前 k 个小写字母，有 q 次询问，每次询问给定一个字符集合 T ，求将 s 中在 T 内的字符去掉后，剩下的字符顺次连接而成的字符串的美丽值。
- $1 \leq n \leq 2 \times 10^5, 1 \leq k \leq 18, 1 \leq q \leq 262143$ 。

- 将极长连续段的平方和转化为，选出两个相同的字符，满足这两个字符之间没有其它字符，求选择的方案数。

- 将极长连续段的平方和转化为，选出两个相同的字符，满足这两个字符之间没有其它字符，求选择的方案数。
- 朴素的想法是枚举两个字符 s_x, s_y 满足 $s_x = s_y = c$ ，若 $[x, y]$ 之间的字符集合为 S ，则当删去的字符集合 T 满足 $c \notin T$ 且 $S \setminus \{c\} \subseteq T$ 时，这对 (x, y) 就有贡献。

- 将极长连续段的平方和转化为，选出两个相同的字符，满足这两个字符之间没有其它字符，求选择的方案数。
- 朴素的想法是枚举两个字符 s_x, s_y 满足 $s_x = s_y = c$ ，若 $[x, y]$ 之间的字符集合为 S ，则当删去的字符集合 T 满足 $c \notin T$ 且 $S \setminus \{c\} \subseteq T$ 时，这对 (x, y) 就有贡献。
- 具体地，设 f_S 表示当删去的字符集合为 S 时的答案，则需要 $\forall S \setminus \{c\}, S \subseteq T$ ，将 f_T 加 1；再 $\forall S \subseteq T$ ，将 f_T 减 1。这个可以通过先算出 f 的高维差分数组，最后再高维前缀和还原出 f 数组即可。

- 将极长连续段的平方和转化为，选出两个相同的字符，满足这两个字符之间没有其它字符，求选择的方案数。
- 朴素的想法是枚举两个字符 s_x, s_y 满足 $s_x = s_y = c$ ，若 $[x, y]$ 之间的字符集合为 S ，则当删去的字符集合 T 满足 $c \notin T$ 且 $S \setminus \{c\} \subseteq T$ 时，这对 (x, y) 就有贡献。
- 具体地，设 f_S 表示当删去的字符集合为 S 时的答案，则需要 $\forall S \setminus \{c\}, S \subseteq T$ ，将 f_T 加 1；再 $\forall S \subseteq T$ ，将 f_T 减 1。这个可以通过先算出 f 的高维差分数组，最后再高维前缀和还原出 f 数组即可。
- 然后发现假设令 $x \leq y$ ，枚举完 y 后， $[x, y]$ 之间的字符集合 S 只有 $\Theta(k)$ 种，所以 $\Theta(nk)$ 计算 f 的高维差分数组即可。

- 将极长连续段的平方和转化为，选出两个相同的字符，满足这两个字符之间没有其它字符，求选择的方案数。
- 朴素的想法是枚举两个字符 s_x, s_y 满足 $s_x = s_y = c$ ，若 $[x, y]$ 之间的字符集合为 S ，则当删去的字符集合 T 满足 $c \notin T$ 且 $S \setminus \{c\} \subseteq T$ 时，这对 (x, y) 就有贡献。
- 具体地，设 f_S 表示当删去的字符集合为 S 时的答案，则需要 $\forall S \setminus \{c\}, S \subseteq T$ ，将 f_T 加 1；再 $\forall S \subseteq T$ ，将 f_T 减 1。这个可以通过先算出 f 的高维差分数组，最后再高维前缀和还原出 f 数组即可。
- 然后发现假设令 $x \leq y$ ，枚举完 y 后， $[x, y]$ 之间的字符集合 S 只有 $\Theta(k)$ 种，所以 $\Theta(nk)$ 计算 f 的高维差分数组即可。
- 设读入总字符个数为 Q ，时间复杂度 $\Theta(nk + 2^k k + Q)$ 。

快速沃尔什变换

定义 11.0.1 (位运算卷积)

定义两个幂级数 A, B 的位运算卷积为：

$$[x^k]C(x) = \sum_{i \oplus j = k} [x^i]A(x)[x^j]B(x)$$

其中 $\oplus$ 是某种位运算，记作卷积式 $A * B = C$ 。

定义 11.0.1 (位运算卷积)

定义两个幂级数 A, B 的位运算卷积为：

$$[x^k]C(x) = \sum_{i \oplus j = k} [x^i]A(x)[x^j]B(x)$$

其中 $\oplus$ 是某种位运算，记作卷积式 $A * B = C$ 。

- FWT (Fast Walsh-Hadamard Transform, 快速沃尔什变换)，用于求解两个幂级数 A, B 的位运算卷积。

定义 11.0.1 (位运算卷积)

定义两个幂级数 A, B 的位运算卷积为：

$$[x^k]C(x) = \sum_{i \oplus j = k} [x^i]A(x)[x^j]B(x)$$

其中 $\oplus$ 是某种位运算，记作卷积式 $A * B = C$ 。

- FWT (Fast Walsh-Hadamard Transform, 快速沃尔什变换)，用于求解两个幂级数 A, B 的位运算卷积。
- 通常 A, B 的项数都会被补全到 2 的幂次，这里设 A, B 都有 2^n 项，从 0 次项一直到 $2^n - 1$ 次项。

- FFT 是将多项式转成点值之后，从卷积变成了点积。

- FFT 是将多项式转成点值之后，从卷积变成了点积。
- 而 FWT 的思想也是将卷积转化为点积。

- FFT 是将多项式转成点值之后，从卷积变成了点积。
- 而 FWT 的思想也是将卷积转化为点积。
- 设 $\text{FWT}(A)$ 表示对 A 做 FWT 变换后得到的幂级数。

- FFT 是将多项式转成点值之后，从卷积变成了点积。
- 而 FWT 的思想也是将卷积转化为点积。
- 设 $\text{FWT}(A)$ 表示对 A 做 FWT 变换后得到的幂级数。
- 我们需要满足 $A * B = C \iff \text{FWT}(A) \cdot \text{FWT}(B) = \text{FWT}(C)$ （点积）。

- FFT 是将多项式转成点值之后，从卷积变成了点积。
- 而 FWT 的思想也是将卷积转化为点积。
- 设 $\text{FWT}(A)$ 表示对 A 做 FWT 变换后得到的幂级数。
- 我们需要满足 $A * B = C \iff \text{FWT}(A) \cdot \text{FWT}(B) = \text{FWT}(C)$ （点积）。
- FFT 是一个线性变换，我们希望 FWT 也是一个线性变换。

- FFT 是将多项式转成点值之后，从卷积变成了点积。
- 而 FWT 的思想也是将卷积转化为点积。
- 设 $\text{FWT}(A)$ 表示对 A 做 FWT 变换后得到的幂级数。
- 我们需要满足 $A * B = C \iff \text{FWT}(A) \cdot \text{FWT}(B) = \text{FWT}(C)$ (点积)。
- FFT 是一个线性变换，我们希望 FWT 也是一个线性变换。
- 设 $c_{i,j}$ 表示从 $[x^i]A(x)$ 到 $[x^j]\text{FWT}(A)$ 所乘的系数。

- $$[x^j]\text{FWT}(A) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x)$$

- $$[x^j]\text{FWT}(A) = \sum_{i=0}^{2^n-1} c_{i,j} [x^i]A(x)$$
- 由于 $[x^i]\text{FWT}(A) \times [x^i]\text{FWT}(B) = [x^i]\text{FWT}(C)$ 。

- $$[x^j]\text{FWT}(A) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x)$$

- 由于 $[x^i]\text{FWT}(A) \times [x^i]\text{FWT}(B) = [x^i]\text{FWT}(C)$ 。

- $$\left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x) \right) \times \left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]B(x) \right) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]C(x)$$

- $$[x^j]\text{FWT}(A) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x)$$

- 由于 $[x^i]\text{FWT}(A) \times [x^i]\text{FWT}(B) = [x^i]\text{FWT}(C)$ 。

- $$\left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x) \right) \times \left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]B(x) \right) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]C(x)$$

- 因为 $[x^i]C(x) = \sum_{j \oplus k = i} [x^j]A(x)[x^k]B(x)$ 。

-

$$[x^j]\text{FWT}(A) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x)$$

- 由于 $[x^i]\text{FWT}(A) \times [x^i]\text{FWT}(B) = [x^i]\text{FWT}(C)$ 。

-

$$\left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]A(x) \right) \times \left(\sum_{i=0}^{2^n-1} c_{i,j}[x^i]B(x) \right) = \sum_{i=0}^{2^n-1} c_{i,j}[x^i]C(x)$$

- 因为 $[x^i]C(x) = \sum_{j \oplus k = i} [x^j]A(x)[x^k]B(x)$ 。

- 将 $[x^i]C(x)$ 代换得：

$$\sum_{i=0}^{2^n-1} \sum_{j=0}^{2^n-1} [x^i]A(x)[x^j]B(x)c_{i,k}c_{j,k} = \sum_{i=0}^{2^n-1} \sum_{j=0}^{2^n-1} [x^i]A(x)[x^j]B(x)c_{i \oplus j, k} \quad \text{对于任意幂级数}$$

A, B 都成立。

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k} c_{j,k} = c_{i \oplus j, k}$ 即可。

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k} c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的 $c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0} c_{i_1,j_1} \dots$ 。

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k} c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的

$c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0} c_{i_1,j_1} \dots$

- 我们也可以将 c 看作一个矩阵：
$$\begin{bmatrix} c_{0,0} & c_{1,0} \\ c_{0,1} & c_{1,1} \end{bmatrix}$$

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k} c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的

$c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0} c_{i_1,j_1} \dots$

- 我们也可以将 c 看作一个矩阵：
$$\begin{bmatrix} c_{0,0} & c_{1,0} \\ c_{0,1} & c_{1,1} \end{bmatrix}$$
- 假如现在已经构造出了符合条件的矩阵 c ，那么如何快速将 A 变换为 $\text{FWT}(A)$ 呢？

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k} c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的

$c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0} c_{i_1,j_1} \dots$

- 我们也可以将 c 看作一个矩阵：
$$\begin{bmatrix} c_{0,0} & c_{1,0} \\ c_{0,1} & c_{1,1} \end{bmatrix}$$
- 假如现在已经构造出了符合条件的矩阵 c ，那么如何快速将 A 变换为 $\text{FWT}(A)$ 呢？
-

$$[x^j] \text{FWT}(A) = \sum_{j=0}^{2^n-1} c_{j,i} [x^j] A(x)$$

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k}c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的

$c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0}c_{i_1,j_1} \dots$

- 我们也可以将 c 看作一个矩阵：
$$\begin{bmatrix} c_{0,0} & c_{1,0} \\ c_{0,1} & c_{1,1} \end{bmatrix}$$
- 假如现在已经构造出了符合条件的矩阵 c ，那么如何快速将 A 变换为 $\text{FWT}(A)$ 呢？

•

$$[x^i]\text{FWT}(A) = \sum_{j=0}^{2^n-1} c_{j,i}[x^j]A(x)$$

- 我们将 $0 \sim 2^n - 1$ 按照最高位拆成两半，设 i' 表示 i 去掉最高位后剩下的数。

- 也就是说，系数 c 只需要满足 $\forall i, j, k \in [0, 2^n), c_{i,k}c_{j,k} = c_{i \oplus j, k}$ 即可。
- 由于位运算的优秀性质，每一位是可以独立开来的。设 m 的二进制表示每一位分别为 $m_0, m_1, m_2, \dots$ ，假设我们已经构造出了符合条件的

$c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ ，那么 $c_{i,j} = c_{i_0,j_0}c_{i_1,j_1} \dots$

- 我们也可以将 c 看作一个矩阵：

$$\begin{bmatrix} c_{0,0} & c_{1,0} \\ c_{0,1} & c_{1,1} \end{bmatrix}$$
- 假如现在已经构造出了符合条件的矩阵 c ，那么如何快速将 A 变换为 $\text{FWT}(A)$ 呢？

•

$$[x^i]\text{FWT}(A) = \sum_{j=0}^{2^n-1} c_{j,i}[x^j]A(x)$$

- 我们将 $0 \sim 2^n - 1$ 按照最高位拆成两半，设 i' 表示 i 去掉最高位后剩下的数。

•

$$[x^i]\text{FWT}(A) = c_{0,i_{n-1}} \sum_{j=0}^{2^{n-1}-1} c_{j,i'}[x^j]A(x) + c_{1,i_{n-1}} \sum_{j=2^{n-1}}^{2^n-1} c_{j,i'}[x^j]A(x)$$

- 容易发现 $\sum_{j=0}^{2^{n-1}-1} c_{j,i'}[x^j]A(x)$ 和 $\sum_{j=2^{n-1}}^{2^n-1} c_{j,i'}[x^j]A(x)$ 都是一个子问题。

FWT 概论

- 容易发现 $\sum_{j=0}^{2^{n-1}-1} c_{j,i'}[x^j]A(x)$ 和 $\sum_{j=2^{n-1}}^{2^n-1} c_{j,i'}[x^j]A(x)$ 都是一个子问题。
- 于是有如下代码：

```
void fwt(int *f) {  
    for (int mid = 1; mid < 1 << n; mid <= 1) {  
        for (int x = 0; x < 1 << n; x += mid << 1) {  
            for (int y = 0; y < mid; ++y) {  
                int t0 = f[x | y], t1 = f[x | y | mid];  
                f[x | y] = c[0][0] * t0 + c[1][0] * t1;  
                f[x | y | mid] = c[0][1] * t0 + c[1][1] * t1;  
            }  
        }  
    }  
}
```

FWT 概论

- 容易发现 $\sum_{j=0}^{2^{n-1}-1} c_{j,i'}[x^j]A(x)$ 和 $\sum_{j=2^{n-1}}^{2^n-1} c_{j,i'}[x^j]A(x)$ 都是一个子问题。
- 于是有如下代码：

```
void fwt(int *f) {  
    for (int mid = 1; mid < 1 << n; mid <= 1) {  
        for (int x = 0; x < 1 << n; x += mid << 1) {  
            for (int y = 0; y < mid; ++y) {  
                int t0 = f[x | y], t1 = f[x | y | mid];  
                f[x | y] = c[0][0] * t0 + c[1][0] * t1;  
                f[x | y | mid] = c[0][1] * t0 + c[1][1] * t1;  
            }  
        }  
    }  
}
```

- 时间复杂度 $\Theta(2^n n)$ 。

- 至于 FWT 逆变换，只要求出 c 矩阵的逆矩阵 c' ，然后用 c' 变换回去就好了。

- 至于 FWT 逆变换，只要求出 c 矩阵的逆矩阵 c' ，然后用 c' 变换回去就好了。
- 所以值得注意的是，构造出来的 c 矩阵必须要有逆矩阵，不然就变不回去了。

- $c_{i,k} c_{j,k} = c_{i \text{ or } j, k}$

- $c_{i,k}c_{j,k} = c_{i \text{ or } j,k}$

- 容易构造出一组符合条件的解 $c = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $c' = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ 。

- $c_{i,k}c_{j,k} = c_{i \text{ or } j,k}$
- 容易构造出一组符合条件的解 $c = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $c' = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ 。
- 在这组解中，FWT 与高维前缀和等价，IFWT 与高维前缀差分等价。

- $c_{i,k}c_{j,k} = c_{i \text{ and } j, k}$

- $c_{i,k}c_{j,k} = c_{i \text{ and } j, k}$
- 容易构造出一组符合条件的解 $c = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $c' = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$ 。

- $c_{i,k}c_{j,k} = c_{i \text{ and } j, k}$
- 容易构造出一组符合条件的解 $c = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $c' = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$ 。
- 在这组解中，FWT 与高维后缀和等价，IFWT 与高维后缀差分等价。

- $C_{i,k} C_{j,k} = C_{i \oplus j, k}$

- $C_{i,k}C_{j,k} = C_{i \text{ xor } j,k}$
- 容易构造出一组符合条件的解 $C = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $C' = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix}$ 。

- $C_{i,k}C_{j,k} = C_{i \text{ xor } j,k}$ 。
- 容易构造出一组符合条件的解 $c = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $c' = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix}$ 。
- 在这组解中, $[x^T]\text{FWT}(A) = \sum_{S=0}^{2^n-1} (-1)^{|S \cap T|} [x^S]A(x)$ 。

- 这里写出原题面的一般形式：
- 你有 n 个 m 元组，其中第 i 个 m 元组为 $(a_{i,1}, a_{i,2}, \dots, a_{i,m})$ ， $\forall i \in [1, n], j \in [1, m]$ ，满足 $a_{i,j} \in [0, 2^k)$ 。
- 有一个长度为 m 的序列 $b_1, b_2, \dots, b_m$ ，你又填充了 n 个数组，其中第 i 个数组中包含 b_1 个 $a_{i,1}$ ， b_2 个 $a_{i,2}$ ，……， b_m 个 $a_{i,m}$ ，每个数组长为 $\sum b_i$ 。
- 你需要在每个数组中选出一个整数，使得它们的异或值为 t ， $\forall t \in [0, 2^k)$ ，求选择的方案数对 998244353 取模后的结果。
- $1 \leq n \leq 10^5, 1 \leq k \leq 17, 0 \leq b_i \leq 10^9, m = 3$ 。

- 考虑 n 个长为 2^k 的数组 $f_1, \dots, f_n$, 其中第 i 个数组满足 $\forall j \in [1, m], f_{i, a_{i,j}} = b_j$ 。

- 考虑 n 个长为 2^k 的数组 $f_1, \dots, f_n$ ，其中第 i 个数组满足 $\forall j \in [1, m], f_{i, a_{i,j}} = b_j$ 。
- 那么我们要求的其实就是这 n 个数组的异或卷积。

- 考虑 n 个长为 2^k 的数组 $f_1, \dots, f_n$, 其中第 i 个数组满足 $\forall j \in [1, m], f_{i, a_{i,j}} = b_j$ 。
- 那么我们要求的其实就是这 n 个数组的异或卷积。
- 考虑对每个数组做一个 FWT, 得到 n 个数组 $g_1, \dots, g_n$ 。那么第 i 个数组满足 $g_{i,S} = \sum_{j=1}^m (-1)^{|S \cap a_{i,j}|} b_j$ 。

- 考虑 n 个长为 2^k 的数组 $f_1, \dots, f_n$, 其中第 i 个数组满足 $\forall j \in [1, m], f_{i, a_{i,j}} = b_j$ 。
- 那么我们要求的其实就是这 n 个数组的异或卷积。
- 考虑对每个数组做一个 FWT, 得到 n 个数组 $g_1, \dots, g_n$ 。那么第 i 个数组满足 $g_{i,S} = \sum_{j=1}^m (-1)^{|S \cap a_{i,j}|} b_j$ 。
- 现在只需要对每个位置 S , 求出 $\prod_{i=1}^n g_{i,S}$, 最后再做一遍 IFWT 即可。

- 考虑 n 个长为 2^k 的数组 $f_1, \dots, f_n$ ，其中第 i 个数组满足 $\forall j \in [1, m], f_{i, a_{i,j}} = b_j$ 。
- 那么我们要求的其实就是这 n 个数组的异或卷积。
- 考虑对每个数组做一个 FWT，得到 n 个数组 $g_1, \dots, g_n$ 。那么第 i 个数组满足 $g_{i,S} = \sum_{j=1}^m (-1)^{|S \cap a_{i,j}|} b_j$ 。
- 现在只需要对每个位置 S ，求出 $\prod_{i=1}^n g_{i,S}$ ，最后再做一遍 IFWT 即可。
- 发现根据 b_j 前面 ± 1 的系数不同，本质不同的 $g_{i,S}$ 只有 2^m 个。将 -1 设为 0 ，将 1 设为 1 ，那么每一种 ± 1 的状态能唯一对应到一个二进制数，现在我们需要求出一个长为 2^m 的数组 c ，其中 c_T 表示在有多少个 i 满足 $g_{i,S}$ 是 T 状态的。

- 我们再处理一个 2^m 个长度为 2^k 的数组，从 $1 \sim n$ 枚举 i ，对于第 S 个数组 h_S ，记 $w = \bigoplus_{j=1}^m [S \text{ 的第 } j \text{ 位为 } 1] a_{i,j}$ ，将 $h_{S,w}$ 加上 1。

- 我们再处理一个 2^m 个长度为 2^k 的数组，从 $1 \sim n$ 枚举 i ，对于第 S 个数组 h_S ，记 $w = \bigoplus_{j=1}^m [S \text{ 的第 } j \text{ 位为 } 1] a_{i,j}$ ，将 $h_{S,w}$ 加上 1。
- 然后把 2^m 个数组都进行 FWT 变换。

- 我们再处理一个 2^m 个长度为 2^k 的数组，从 $1 \sim n$ 枚举 i ，对于第 S 个数组 h_S ，记 $w = \bigoplus_{j=1}^m [S \text{ 的第 } j \text{ 位为 } 1] a_{i,j}$ ，将 $h_{S,w}$ 加上 1。
- 然后把 2^m 个数组都进行 FWT 变换。
- 注意到这 2^m 个数组的第 S 位上的值，对应了关于 c 数组的 2^m 个方程，由于系数有规律，直接消元即可。

- 我们再处理一个 2^m 个长度为 2^k 的数组，从 $1 \sim n$ 枚举 i ，对于第 S 个数组 h_S ，记 $w = \bigoplus_{j=1}^m [S \text{ 的第 } j \text{ 位为 } 1] a_{i,j}$ ，将 $h_{S,w}$ 加上 1。
- 然后把 2^m 个数组都进行 FWT 变换。
- 注意到这 2^m 个数组的第 S 位上的值，对应了关于 c 数组的 2^m 个方程，由于系数有规律，直接消元即可。
- 时间复杂度 $\Theta((m+k)2^{m+k} + nm2^m)$ 。

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 其实就是只有 $S \cap T = \emptyset$ 时 $a_S b_T$ 才能贡献到 $c_{S \cup T}$ 。

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 其实就是只有 $S \cap T = \emptyset$ 时 $a_S b_T$ 才能贡献到 $c_{S \cup T}$ 。
- 而 $S \cap T = \emptyset$ 又等价于 $|S| + |T| = |S \cup T|$ 。

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 其实就是只有 $S \cap T = \emptyset$ 时 $a_S b_T$ 才能贡献到 $c_{S \cup T}$ 。
- 而 $S \cap T = \emptyset$ 又等价于 $|S| + |T| = |S \cup T|$ 。
- 直接将数组 a, b, c 扩展成一个 $n+1$ 次多项式 A, B, C 即可, 其中 $A_S = a_S x^{|S|}$, B 以此类推。

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 其实就是只有 $S \cap T = \emptyset$ 时 $a_S b_T$ 才能贡献到 $c_{S \cup T}$ 。
- 而 $S \cap T = \emptyset$ 又等价于 $|S| + |T| = |S \cup T|$ 。
- 直接将数组 a, b, c 扩展成一个 $n+1$ 次多项式 A, B, C 即可, 其中 $A_S = a_S x^{|S|}$, B 以此类推。
- 对多项式数组 A, B 做 OR 卷积, 得到数组 C 后, $c_S = [x^{|S|}]C_S$ 。

- 给定数组 $a_0, a_1, \dots, a_{2^n-1}$ 和 $b_0, b_1, \dots, b_{2^n-1}$, 求数组 c , 满足:

$$c_T = \sum_{S \subseteq T} a_S b_{T \setminus S}$$

- 其实就是只有 $S \cap T = \emptyset$ 时 $a_S b_T$ 才能贡献到 $c_{S \cup T}$ 。
- 而 $S \cap T = \emptyset$ 又等价于 $|S| + |T| = |S \cup T|$ 。
- 直接将数组 a, b, c 扩展成一个 $n + 1$ 次多项式 A, B, C 即可, 其中 $A_S = a_S x^{|S|}$, B 以此类推。
- 对多项式数组 A, B 做 OR 卷积, 得到数组 C 后, $c_S = [x^{|S|}]C_S$ 。
- 时间复杂度 $\Theta(2^n n^2)$ 。

- 有 n 座城市，第 i 座城市的人口为 w_i ，城市与城市之间可能有双向道路相连。小 S 要将这 n 座城市划分成若干个州，每个州至少有一座城市。
- 假设小 S 将 n 座城市划分成了 k 个州，设 V_i 是第 i 个州包含的所有城市组成的集合。定义一个州是不合法的，当且仅当这个州的导出子图联通并且存在欧拉回路，反之这个州是合法的。
- 定义第 i 个州的满意度为：第 i 个州的人口在前 i 个州的人口中所占比例的 p 次幂，即
$$\left(\frac{\sum_{x \in V_i} w_x}{\sum_{j=1}^i \sum_{x \in V_j} w_x} \right)^p。$$
- 定义一个划分的满意度为所有州的满意度的乘积；定义一个划分是合法的，当且仅当其中所有州都是合法的。
- 求所有合法划分的满意度之和，答案对 998244353 取模，两个划分 $\{V_1, V_2, \dots, V_k\}$ 和 $\{C_1, C_2, \dots, C_s\}$ 是不同的，当且仅当 $k \neq s$ ，或存在某个 $1 \leq i \leq k$ ，使得 $V_i \neq C_i$ 。
- $0 \leq n \leq 21, 0 \leq m \leq n(n-1)/2, 0 \leq p \leq 2, 1 \leq w_i \leq 100$ ，时限 15s。

- 预处理出每个集合 S 是否合法，记做 $a_S = [S \text{合法}]$ ，以及其大小的 p 次幂，记做 w_S ，以及贡献 $w'_S = a_S w_S$ 。

- 预处理出每个集合 S 是否合法，记做 $a_S = [S \text{合法}]$ ，以及其大小的 p 次幂，记做 W_S ，以及贡献 $W'_S = a_S W_S$ 。
- 设 f_S 表示当前已经被划分的城市的集合为 S ，枚举 S_0 表示当前划分里最后一个州，则有：
$$f_S = \frac{1}{W_S} \sum_{S_0 \subset S} f_{S_0} W'_{S \setminus S_0}。$$

- 预处理出每个集合 S 是否合法，记做 $a_S = [S \text{合法}]$ ，以及其大小的 p 次幂，记做 W_S ，以及贡献 $W'_S = a_S W_S$ 。
- 设 f_S 表示当前已经被划分的城市的集合为 S ，枚举 S_0 表示当前划分里最后一个州，则有：
$$f_S = \frac{1}{W_S} \sum_{S_0 \subset S} f_{S_0} W'_{S \setminus S_0}。$$
- 半在线子集卷积优化即可，时间复杂度 $\Theta(2^n n^2)$ 。

集合幂级数

- 与生成函数类似，设全集为 U ，我们可以设幂级数 $F(x) = \sum_{S \subseteq U} a_S x^S$ 。

- 与生成函数类似，设全集为 U ，我们可以设幂级数 $F(x) = \sum_{S \subseteq U} a_S x^S$ 。
- 集合幂级数的乘法定义为 and 卷积，or 卷积，xor 卷积，子集卷积之类。

集合幂级数引入

- 与生成函数类似，设全集为 U ，我们可以设幂级数 $F(x) = \sum_{S \subseteq U} a_S x^S$ 。
- 集合幂级数的乘法定义为 and 卷积，or 卷积，xor 卷积，子集卷积之类。
- 集合幂级数也能够进行求逆，ln，exp，直接 FWT 后转多项式的操作，然后 IFFT 即可。

集合幂级数引入

- 与生成函数类似，设全集为 U ，我们可以设幂级数 $F(x) = \sum_{S \subseteq U} a_S x^S$ 。
- 集合幂级数的乘法定义为 and 卷积，or 卷积，xor 卷积，子集卷积之类。
- 集合幂级数也能够进行求逆，ln，exp，直接 FWT 后转多项式的操作，然后 IFFT 即可。
- 有些题目中会对集合幂级数进行 FWT，IFWT 等操作。

- 给定一个长度为 n 的非负整数序列 $A = \{a_1, a_2, \dots, a_n\}$, 对于 A 的一个子序列 $B = \{a_{b_1}, a_{b_2}, \dots, a_{b_m}\}$ ($0 \leq m \leq n$, $1 \leq b_1 < b_2 < \dots < b_m \leq n$, 下同), 称 B 是 A 的优秀子序列当且仅当, 其任意两个不同元素的按位与结果均为 0, 即: $\forall 1 \leq i < j \leq m$, 满足: $a_{b_i} \text{ and } a_{b_j} = 0$, 其中 and 是按位与运算。
- 对于子序列 $B = \{a_{b_1}, a_{b_2}, \dots, a_{b_m}\}$, 我们定义其价值为 $\varphi(1 + \sum_{i=1}^m a_{b_i})$, 其中 $\varphi(x)$ 表示小等于 x 的正整数中与 x 互质的数的个数。
- 现在请你求出 A 的所有优秀子序列的价值之和, 答案对 $10^9 + 7$ 取模。
- $1 \leq n \leq 10^6, 0 \leq a_i \leq 2 \times 10^5$ 。

- 先把 $a_i = 0$ 的去掉，最后答案乘上一个 2 的幂次即可。

- 先把 $a_i = 0$ 的去掉，最后答案乘上一个 2 的幂次即可。
- 然后设形式幂级数 $F(x) = \prod_{i=1}^n (1 + x^{a_i})$ ，定义乘法为子集卷积。

- 先把 $a_i = 0$ 的去掉，最后答案乘上一个 2 的幂次即可。
- 然后设形式幂级数 $F(x) = \prod_{i=1}^n (1 + x^{a_i})$ ，定义乘法为子集卷积。
- 则 $[x^S]F(x)$ 表示当 $\sum a_{b_i} = S$ 时，优秀子序列的方案数。

- 先把 $a_i = 0$ 的去掉，最后答案乘上一个 2 的幂次即可。
- 然后设形式幂级数 $F(x) = \prod_{i=1}^n (1 + x^{a_i})$ ，定义乘法为子集卷积。
- 则 $[x^S]F(x)$ 表示当 $\sum a_{b_i} = S$ 时，优秀子序列的方案数。
- 只要能把 $F(x)$ 的各项系数求出即可，对于 n 个形式幂级数，发现做 FWT 非 0 项有 m 项；但是做 $\ln$ 非 0 项只有 1 项。

- 先把 $a_i = 0$ 的去掉，最后答案乘上一个 2 的幂次即可。
- 然后设形式幂级数 $F(x) = \prod_{i=1}^n (1 + x^{a_i})$ ，定义乘法为子集卷积。
- 则 $[x^S]F(x)$ 表示当 $\sum a_{b_i} = S$ 时，优秀子序列的方案数。
- 只要能把 $F(x)$ 的各项系数求出即可，对于 n 个形式幂级数，发现做 FWT 非 0 项有 m 项；但是做 $\ln$ 非 0 项只有 1 项。
- 于是对 n 个形式幂级数做 $\ln$ ，加起来后做 $\exp$ 即可。时间复杂度 $\Theta(n + m \log^2 m)$ 。

2023-2024 集训队互测 Round 12 C. goods

- 给定 n 个物品，定义第 i 个物品的特征值为 a_i ，其中 a_i 是整数且满足 $0 \leq a_i < 2^m$ 。
- 你需要选择若干物品形成一个组合。一个组合的**独特值**为其包含的物品的特征值的**异或和**。注意，你可以一个物品也不选，此时我们认为独特值为 0。
- 当你选择出一个组合后，你会邀请你的两个伙伴来挑选这些物品。只不过，两个人只约定了一共选出 B 个物品，而忘记保证它们互不冲突，也就是说一个物品可能同时被两人选择。**请注意**，一共选出 B 个物品指的是选择次数为 B ，而不是被选择的物品数为 B 。
- 你对一个组合产生的**期待值**为两个伙伴从组合中挑选物品的本质不同方案数。两个方案本质不同当且仅当存在一个物品，使得其在第一个方案中被某个人选择了，而在第二个方案中没有被那个人选择。对于每个 $0 \leq i < 2^m$ ，你需要对所有独特值为 i 的组合，求出它们的期待值之和。
- 由于答案可能过大，你只需要输出答案对 998244353 取模的结果。
- $1 \leq n \leq 10^6, 1 \leq m \leq 20, 0 \leq B \leq n$ 。

- 定义二元生成函数 $F(x) = \prod_{i=1}^n (1 + x^{a_i}(1 + 2y + y^2))$ ，其中 x 这维是异或卷积， y 这维是加法卷积。

- 定义二元生成函数 $F(x) = \prod_{i=1}^n (1 + x^{a_i}(1 + 2y + y^2))$ ，其中 x 这维是异或卷积， y 这维是加法卷积。
- 这是 n 个多项式相乘的结果，对于每个多项式进行 FWT，消去 x 元，然后得到在 S 位上关于 y 的多项式是 $\prod_{i=1}^n (1 + (-1)^{|S \cap a_i|}(1 + 2y + y^2))$ 。

- 定义二元生成函数 $F(x) = \prod_{i=1}^n (1 + x^{a_i}(1 + 2y + y^2))$ ，其中 x 这维是异或卷积， y 这维是加法卷积。
- 这是 n 个多项式相乘的结果，对于每个多项式进行 FWT，消去 x 元，然后得到在 S 位上关于 y 的多项式是 $\prod_{i=1}^n (1 + (-1)^{|S \cap a_i|} (1 + 2y + y^2))$ 。
- 对于每个位 S ，可以简单求出有 t 个 a_i 满足 $|S \cap a_i|$ 为偶数，有 $n - t$ 个为奇数。然后只要求 $[y^B](2 + 2y + y^2)^t (-2y - y^2)^{n-t}$ 。

- 定义二元生成函数 $F(x) = \prod_{i=1}^n (1 + x^{a_i}(1 + 2y + y^2))$ ，其中 x 这维是异或卷积， y 这维是加法卷积。
- 这是 n 个多项式相乘的结果，对于每个多项式进行 FWT，消去 x 元，然后得到在 S 位上关于 y 的多项式是 $\prod_{i=1}^n (1 + (-1)^{|S \cap a_i|}(1 + 2y + y^2))$ 。
- 对于每个位 S ，可以简单求出有 t 个 a_i 满足 $|S \cap a_i|$ 为偶数，有 $n - t$ 个为奇数。然后只需要求 $[y^B](2 + 2y + y^2)^t(-2y - y^2)^{n-t}$ 。
-

$$\begin{aligned}
 & [y^B](2 + 2y + y^2)^t(-2y - y^2)^{n-t} \\
 &= [y^{B-n+t}](-1)^{n-t}(2 + 2y + y^2)^t(2 + y)^{n-t} \\
 &= (-1)^{n-t} \sum_{i=0}^t \binom{t}{i} 2^{t-i} [y^{B-n+t-i}](2 + y)^{n-t+i} \\
 &= (-1)^{n-t} \sum_{i=0}^t \binom{t}{i} 2^{t-i} \binom{n-t+i}{B-n+t-i} 2^{2n-B-2t+2i}
 \end{aligned}$$

- 设 H_t 等于上面的式子，发现可以 NTT 预处理。

- 设 H_t 等于上面的式子，发现可以 NTT 预处理。
- 最后做一遍 IFWT 即可，时间复杂度 $\Theta(n \log n + m2^m)$ 。

拉格朗日插值

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。
- 拉格朗日插值可以通过这 $n + 1$ 个点推出原来的多项式。

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。
- 拉格朗日插值可以通过这 $n + 1$ 个点推出原来的多项式。
- 设 $n + 1$ 个点分别为 $(x_0, y_0), \dots, (x_n, y_n)$ ，原多项式为 $F(x)$ 。

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。
- 拉格朗日插值可以通过这 $n + 1$ 个点推出原来的多项式。
- 设 $n + 1$ 个点分别为 $(x_0, y_0), \dots, (x_n, y_n)$ ，原多项式为 $F(x)$ 。
-

$$F(x) = \sum_{i=0}^n y_i \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$$

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。
- 拉格朗日插值可以通过这 $n + 1$ 个点推出原来的多项式。
- 设 $n + 1$ 个点分别为 $(x_0, y_0), \dots, (x_n, y_n)$ ，原多项式为 $F(x)$ 。
-

$$F(x) = \sum_{i=0}^n y_i \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$$

- 考虑 $\prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$ ，当 $x = x_i$ 时值为 1，当 $x = x_j (j \neq i)$ 时值为 0，所以上述多项式经过 $(x_0, y_0), \dots, (x_n, y_n)$ 。

- 给定一个 n 次多项式在坐标系上 $n + 1$ 个不同的点，即可唯一确定这个多项式。
- 拉格朗日插值可以通过这 $n + 1$ 个点推出原来的多项式。
- 设 $n + 1$ 个点分别为 $(x_0, y_0), \dots, (x_n, y_n)$ ，原多项式为 $F(x)$ 。
-

$$F(x) = \sum_{i=0}^n y_i \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$$

- 考虑 $\prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$ ，当 $x = x_i$ 时值为 1，当 $x = x_j (j \neq i)$ 时值为 0，所以上述多项式经过 $(x_0, y_0), \dots, (x_n, y_n)$ 。
- 之后的内容假设除法运算的复杂度为 $\Theta(1)$ 。

-

$$F(x) = \sum_{i=0}^n \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)} \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

-

$$F(x) = \sum_{i=0}^n \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)} \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 设 $t_i = \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)}$ 。

-

$$F(x) = \sum_{i=0}^n \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)} \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 设 $t_i = \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)}$

-

$$F(x) = \sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

重心拉格朗日插值

-

$$F(x) = \sum_{i=0}^n \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)} \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 设 $t_i = \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)}$ 。

-

$$F(x) = \sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 维护 $(x - x_i)$ 的前缀后缀积，假设我们已经维护好 t ，即可做到 $\Theta(n)$ 单次查询。

重心拉格朗日插值

-

$$F(x) = \sum_{i=0}^n \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)} \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 设 $t_i = \frac{y_i}{\prod_{0 \leq j \leq n, j \neq i} (x_i - x_j)}$ 。

-

$$F(x) = \sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$$

- 维护 $(x - x_i)$ 的前缀后缀积，假设我们已经维护好 t ，即可做到 $\Theta(n)$ 单次查询。
- 每次新加入一个点，对之前点 t 的影响可以 $\Theta(1)$ 修改，对新的点 t 可以 $\Theta(n)$ 计算，时间复杂度 $\Theta(n)$ 。

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。

求多项式各项系数

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。

求多项式各项系数

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。
- 设 $f_{l,r} = \sum_{i=l}^r t_i \prod_{l \leq j \leq r, j \neq i} (x - x_j)$, $g_{l,r} = \prod_{l \leq i \leq r} (x - x_i)$ 。

求多项式各项系数

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。
- 设 $f_{l,r} = \sum_{i=l}^r t_i \prod_{l \leq j \leq r, j \neq i} (x - x_j)$, $g_{l,r} = \prod_{l \leq i \leq r} (x - x_i)$ 。
- 设 $M = (l + r)/2$ ，已经计算出 $f_{l,M}, g_{l,M}, f_{M+1,r}, g_{M+1,r}$ 。

求多项式各项系数

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。
- 设 $f_{l,r} = \sum_{i=l}^r t_i \prod_{l \leq j \leq r, j \neq i} (x - x_j)$, $g_{l,r} = \prod_{l \leq i \leq r} (x - x_i)$ 。
- 设 $M = (l + r)/2$ ，已经计算出 $f_{l,M}, g_{l,M}, f_{M+1,r}, g_{M+1,r}$ 。
-

$$\begin{cases} f_{l,r} = f_{l,M}g_{M+1,r} + g_{l,M}f_{M+1,r} \\ g_{l,r} = g_{l,M}g_{M+1,r} \end{cases}$$

求多项式各项系数

- 直接暴力计算上一页中的 t_i ，时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。
- 设 $f_{l,r} = \sum_{i=l}^r t_i \prod_{l \leq j \leq r, j \neq i} (x - x_j)$, $g_{l,r} = \prod_{l \leq i \leq r} (x - x_i)$ 。
- 设 $M = (l + r)/2$ ，已经计算出 $f_{l,M}, g_{l,M}, f_{M+1,r}, g_{M+1,r}$ 。
-

$$\begin{cases} f_{l,r} = f_{l,M}g_{M+1,r} + g_{l,M}f_{M+1,r} \\ g_{l,r} = g_{l,M}g_{M+1,r} \end{cases}$$

- $T(1) = \Theta(1), T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$ 。

求多项式各项系数

- 直接暴力计算上一页中的 t_i , 时间复杂度 $\Theta(n^2)$ 。
- 考虑分治计算 $\sum_{i=0}^n t_i \prod_{0 \leq j \leq n, j \neq i} (x - x_j)$ 。
- 设 $f_{l,r} = \sum_{i=l}^r t_i \prod_{l \leq j \leq r, j \neq i} (x - x_j)$, $g_{l,r} = \prod_{l \leq i \leq r} (x - x_i)$ 。
- 设 $M = (l + r)/2$, 已经计算出 $f_{l,M}, g_{l,M}, f_{M+1,r}, g_{M+1,r}$ 。
-

$$\begin{cases} f_{l,r} = f_{l,M}g_{M+1,r} + g_{l,M}f_{M+1,r} \\ g_{l,r} = g_{l,M}g_{M+1,r} \end{cases}$$

- $T(1) = \Theta(1), T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$ 。
- 多项式快速求值可以做到 $\Theta(n \log^2 n)$ 。

- 回到开始的式子：

- 回到开始的式子：
-

$$F(x) = \sum_{i=0}^n y_i \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$$

取值连续时的做法

- 回到开始的式子：
-

$$F(x) = \sum_{i=0}^n y_i \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j}$$

- 当 $x_i = i$ 时，维护 $(x - x_j)$ 的前缀后缀积，阶乘逆元，即可直接 $\Theta(n)$ 计算出多项式在某个点的值。

- 给定 n, k , 求:
-

$$\sum_{x=1}^n x^k$$

- 答案对 $10^9 + 7$ 取模。
- $1 \leq n \leq 10^9, 0 \leq k \leq 10^6$ 。

- 由于 $\sum x^k = x^{k+1}/(k+1)$ 。

- 由于 $\sum x^k = x^{k+1}/(k+1)$ 。
- 设 $F(x)$ 为一个 k 次多项式，则 $\sum F(x)$ 为 $k+1$ 次多项式。

- 由于 $\sum x^k = x^{k+1}/(k+1)$ 。
- 设 $F(x)$ 为一个 k 次多项式，则 $\sum F(x)$ 为 $k+1$ 次多项式。
- 所以 $\sum_{x=1}^n x^k$ 为 $k+1$ 次多项式，直接取 $x = 1, \dots, k+1$ 然后 $\Theta(k)$ 拉格朗日插值即可。

- 由于 $\sum x^k = x^{k+1}/(k+1)$ 。
- 设 $F(x)$ 为一个 k 次多项式，则 $\sum F(x)$ 为 $k+1$ 次多项式。
- 所以 $\sum_{x=1}^n x^k$ 为 $k+1$ 次多项式，直接取 $x = 1, \dots, k+1$ 然后 $\Theta(k)$ 拉格朗日插值即可。
- 用线性筛求 x^k 时间复杂度 $\Theta\left(\frac{k \log k}{\ln k}\right) = \Theta(k)$ ，直接快速幂时间复杂度 $\Theta(k \log k)$ 。

- 给定一个函数 fun 的伪代码：

```
function fun(A)
  if A.length > 0
    let L[1 ... L.length] and R[1 ... R.length] be new arrays
    L.length = R.length = 0
    for i = 2 to A.length
      if A[i] < A[1]
        L.length = L.length + 1
        L[L.length] = A[i]
      else
        R.length = R.length + 1
        R[R.length] = A[i]
    return A.length + fun(L) + fun(R)
  else
    return 0
```

- 求有多少个 $1 \sim n$ 的排列 p ，满足 $\text{fun}(p) \geq L$ 。
- $1 \leq n \leq 200, 1 \leq L \leq 10^9$ 。

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。
-

$$F_n(z) = z \sum_{x=0}^{n-1} F_x(z) F_{n-1-x}(z)$$

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。
-

$$F_n(z) = z \sum_{x=0}^{n-1} F_x(z) F_{n-1-x}(z)$$

- 于是 $f_{n,s} = s! [x^s] F_n(z)$ 。

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。
-

$$F_n(z) = z \sum_{x=0}^{n-1} F_x(z) F_{n-1-x}(z)$$

- 于是 $f_{n,s} = s![x^s]F_n(z)$ 。
- 也就是说如果我们求出 $F_n(z)$ 的系数就能得出答案。

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。
-

$$F_n(z) = z \sum_{x=0}^{n-1} F_x(z) F_{n-1-x}(z)$$

- 于是 $f_{n,s} = s! [x^s] F_n(z)$ 。
- 也就是说如果我们求出 $F_n(z)$ 的系数就能得出答案。
- 因为 $F_n(z)$ 是的次数是 $\Theta(n^2)$ 级别的，我们直接代入 $\Theta(n^2)$ 个 z 具体的值，然后计算 $F_n(z)$ 的值，这样一遍是 $\Theta(n^2)$ 的，总时间复杂度就是 $\Theta(n^4)$ 的。

CF1874E Jellyfish and Hack

- 首先有一个 $\Theta(n^6)$ 的暴力 dp。
- 设 $f_{n,s}$ 表示有多少长度为 n 的排列 p 满足 $\text{fun}(p) = s$ 。
-

$$f_{n,s} = \sum_{x=0}^{n-1} \sum_{y=0}^{s-n} \binom{n-1}{x} f_{x,y} f_{n-1-x,s-n-y}$$

- 上个 NTT 可以做到 $\Theta(n^4 \log n)$ ，但是没什么用。
- 考虑设 $F_n(z) = \sum_{s \geq 0} f_{n,s} \frac{z^s}{s!}$ 。
-

$$F_n(z) = z \sum_{x=0}^{n-1} F_x(z) F_{n-1-x}(z)$$

- 于是 $f_{n,s} = s! [x^s] F_n(z)$ 。
- 也就是说如果我们求出 $F_n(z)$ 的系数就能得出答案。
- 因为 $F_n(z)$ 是的次数是 $\Theta(n^2)$ 级别的，我们直接代入 $\Theta(n^2)$ 个 z 具体的值，然后计算 $F_n(z)$ 的值，这样一遍是 $\Theta(n^2)$ 的，总时间复杂度就是 $\Theta(n^4)$ 的。
- 然后再利用拉格朗日插值计算出 $F_n(z)$ 的系数，时间复杂度 $\Theta(n^4)$ 。

- 有一棵 n 个节点的无根树，刚开始树上每个节点的权值均为 0。KK 想对这棵树进行一些修改，他会任选一个节点作为初始的当前节点，然后重复以下动作：
 1. 将当前节点 i 的权值修改为一个正整数 x ，需满足 $l_i \leq x \leq r_i$ 。其中 l_i, r_i 是输入中给出的两个正整数。
 2. 结束修改过程，或移动到一个与当前节点相邻的权值为 0 的节点（如果不存在这样的节点，则必须结束修改过程）。
- 现在 KK 有两个问题：
 1. 在修改结束后，可以得到多少棵不同的树，满足树上非零权值的最大值和最小值的差小于等于 K ？其中 K 是输入中给出的一个正整数。
 2. 这些满足条件的树的权值之和为多少？（树的权值定义为这棵树上所有节点的权值之和）
- 你需要输出这两个问题的答案模 $10^9 + 7$ 。我们认为两棵树不同当且仅当至少存在一个节点的权值不同。
- $1 \leq n \leq 200, 1 \leq l_i \leq r_i \leq 10^9, 1 \leq K \leq 10^9$ 。

- 假设选择了一条长度为 L 的路径，那么方案数就是

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \max(\min(x + K, r_u) - \max(x, l_u) + 1, 0), \text{ 权值和就是}$$

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \left(\sum_{w=\max(l_u, x)}^{\min(r_u, x+K)} w \right), \text{ 容易发现两者都是分段 } \Theta(n) \text{ 次多项式, 段数也是 } \Theta(n) \text{ 的。}$$

- 假设选择了一条长度为 L 的路径，那么方案数就是

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \max(\min(x + K, r_u) - \max(x, l_u) + 1, 0), \text{ 权值和就是}$$

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \left(\sum_{w=\max(l_u, x)}^{\min(r_u, x+K)} w \right), \text{ 容易发现两者都是分段 } \Theta(n) \text{ 次多项式, 段数也是 } \Theta(n) \text{ 的。}$$

- 同理，所有路径方案数之和，与所有路径权值和也是 $\Theta(n)$ 次多项式，段数同样是 $\Theta(n)$ 的。

- 假设选择了一条长度为 L 的路径，那么方案数就是

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \max(\min(x + K, r_u) - \max(x, l_u) + 1, 0), \text{ 权值和就是}$$

$$\sum_{x=0}^{\max r} \prod_{u \in \text{Path}} \left(\sum_{w=\max(l_u, x)}^{\min(r_u, x+K)} w \right), \text{ 容易发现两者都是分段 } \Theta(n) \text{ 次多项式, 段数也是 } \Theta(n) \text{ 的。}$$

- 同理，所有路径方案数之和，与所有路径权值和也是 $\Theta(n)$ 次多项式，段数同样是 $\Theta(n)$ 的。
- 枚举每一段，跑出 $\Theta(n)$ 个点值然后拉格朗日插值即可，每算一次点值需要一次 $\Theta(n)$ 的树形 dp，总时间复杂度 $\Theta(n^3)$ 。

- 给定 n, a, k , 求:
-

$$\sum_{i=1}^n i^k a^i$$

- 答案对 998244353 取模。
- $1 \leq k \leq 10^7, 1 \leq n, a \leq 998244352$ 。

- $a = 1$ 时直接拉格朗日插值，时间复杂度 $\Theta(k)$ ，下面考虑 $a > 1$ 的情况。

- $a = 1$ 时直接拉格朗日插值，时间复杂度 $\Theta(k)$ ，下面考虑 $a > 1$ 的情况。

-

$$\sum_{i=1}^n i^k a^i = \sum_1^{n+1} a^x x^k \delta x$$

- $a = 1$ 时直接拉格朗日插值，时间复杂度 $\Theta(k)$ ，下面考虑 $a > 1$ 的情况。

-

$$\sum_{i=1}^n i^k a^i = \sum_1^{n+1} a^x x^k \delta x$$

- 考虑分部求和公式，设 $u(x) = x^k, \Delta v(x) = a^x, v(x) = \frac{a^x}{a-1}, Ev(x) = \frac{a^{x+1}}{a-1}^\circ$

- $a = 1$ 时直接拉格朗日插值，时间复杂度 $\Theta(k)$ ，下面考虑 $a > 1$ 的情况。

-

$$\sum_{i=1}^n i^k a^i = \sum_1^{n+1} a^x x^k \delta x$$

- 考虑分部求和公式，设 $u(x) = x^k, \Delta v(x) = a^x, v(x) = \frac{a^x}{a-1}, Ev(x) = \frac{a^{x+1}}{a-1}$ 。

-

$$\sum a^x x^k = u(x)v(x) - \sum Ev(x)\Delta u(x)$$

$$= \frac{a^x x^k}{a-1} - \sum \frac{\Delta u(x) a^{x+1}}{a-1}$$

- $a = 1$ 时直接拉格朗日插值，时间复杂度 $\Theta(k)$ ，下面考虑 $a > 1$ 的情况。

-

$$\sum_{i=1}^n i^k a^i = \sum_{x=1}^{n+1} a^x x^k \delta x$$

- 考虑分部求和公式，设 $u(x) = x^k$, $\Delta v(x) = a^x$, $v(x) = \frac{a^x}{a-1}$, $Ev(x) = \frac{a^{x+1}}{a-1}$ 。

-

$$\sum a^x x^k = u(x)v(x) - \sum Ev(x)\Delta u(x)$$

$$= \frac{a^x x^k}{a-1} - \sum \frac{\Delta u(x) a^{x+1}}{a-1}$$

- 设 $\sum a^x x^k = a^x g(x)$ ，其中 $g(x)$ 为 k 次多项式。

- 设 $f(n) = \sum_1^n a^x x^k \delta x$, 答案即为 $f(n+1)$ 。

- 设 $f(n) = \sum_1^n a^x x^k \delta x$, 答案即为 $f(n+1)$ 。

-

$$f(n) = a^x g(x) \Big|_1^n = a^n g(n) - a \cdot g(1)$$

- 设 $f(n) = \sum_1^n a^x x^k \delta x$, 答案即为 $f(n+1)$ 。

-

$$f(n) = a^x g(x) \Big|_1^n = a^n g(n) - a \cdot g(1)$$

- 发现只要求出 $g(1), g(2), \dots, g(k+1)$, 通过拉格朗日插值得出 $g(n+1)$, 就能计算答案了。

- 设 $f(n) = \sum_1^n a^x x^k \delta x$, 答案即为 $f(n+1)$ 。

-

$$f(n) = a^x g(x) \Big|_1^n = a^n g(n) - a \cdot g(1)$$

- 发现只要求出 $g(1), g(2), \dots, g(k+1)$, 通过拉格朗日插值得出 $g(n+1)$, 就能计算答案了。
- 考虑线性筛 $\Theta(k)$ 求出 x^k , 递推求出 $f(1), f(2), \dots, f(k+1)$ 。

- 设 $f(n) = \sum_1^n a^x x^k \delta x$, 答案即为 $f(n+1)$ 。

-

$$f(n) = a^x g(x) \Big|_1^n = a^n g(n) - a \cdot g(1)$$

- 发现只需要求出 $g(1), g(2), \dots, g(k+1)$, 通过拉格朗日插值得出 $g(n+1)$, 就能计算答案了。
- 考虑线性筛 $\Theta(k)$ 求出 x^k , 递推求出 $f(1), f(2), \dots, f(k+1)$ 。
- 由于 $f(n) = a^n g(n) - a \cdot g(1)$, 所以只要求出 $g(1)$ 就能求出 $g(2), \dots, g(k+1)$, 问题转化为求 $g(1)$ 。

- 理论上 $g(1)$ 可以任意钦定，我们能够构造出无穷个符合条件的函数 $g(x)$ ，但是选取不恰当的 $g(1)$ 可能会使得 $g(x)$ 不是 k 次函数。

- 理论上 $g(1)$ 可以任意钦定，我们能够构造出无穷个符合条件的函数 $g(x)$ ，但是选取不恰当的 $g(1)$ 可能会使得 $g(x)$ 不是 k 次函数。
- 对 k 次函数做 $k+1$ 次差分的结果一定为 0，所以可得：

- 理论上 $g(1)$ 可以任意钦定，我们能够构造出无穷个符合条件的函数 $g(x)$ ，但是选取不恰当的 $g(1)$ 可能会使得 $g(x)$ 不是 k 次函数。
- 对 k 次函数做 $k+1$ 次差分的结果一定为 0，所以可得：
-

$$\Delta^{k+1}g(1) = \sum_{i=0}^{k+1} (-1)^{k+1-i} \binom{k+1}{i} g(i+1) = 0$$

- 理论上 $g(1)$ 可以任意钦定，我们能够构造出无穷个符合条件的函数 $g(x)$ ，但是选取不恰当的 $g(1)$ 可能会使得 $g(x)$ 不是 k 次函数。
- 对 k 次函数做 $k+1$ 次差分的结果一定为 0，所以可得：

$$\Delta^{k+1}g(1) = \sum_{i=0}^{k+1} (-1)^{k+1-i} \binom{k+1}{i} g(i+1) = 0$$

- 由于 $g(n), n > 1$ 都可以通过 $g(1)$ 来表示，所以这是一个关于 $g(1)$ 的一元一次方程，解出 $g(1)$ 即可。

- 理论上 $g(1)$ 可以任意钦定，我们能够构造出无穷个符合条件的函数 $g(x)$ ，但是选取不恰当的 $g(1)$ 可能会使得 $g(x)$ 不是 k 次函数。
- 对 k 次函数做 $k+1$ 次差分的结果一定为 0，所以可得：
-

$$\Delta^{k+1}g(1) = \sum_{i=0}^{k+1} (-1)^{k+1-i} \binom{k+1}{i} g(i+1) = 0$$

- 由于 $g(n), n > 1$ 都可以通过 $g(1)$ 来表示，所以这是一个关于 $g(1)$ 的一元一次方程，解出 $g(1)$ 即可。
- 至此，问题解决，时间复杂度 $\Theta(k)$ 。

Good Luck & Have Fun!